Quantstamp

Polymarket V2

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Prediction Market
Timeline	2026-04-24 through 2026-05-06
Language	Solidity
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review
Source Code	• Polymarket/polymarket-v2 #a340369
Auditors	• Paul Clemson Auditing Engineer • Jennifer Wu Auditing Engineer • Tim Sigl Auditing Engineer

Documentation quality	High
Test quality	High
Total Findings	12 Fixed: 7 Acknowledged: 5
High severity findings	1 Fixed: 1
Medium severity findings	1 Fixed: 1
Low severity findings	6 Fixed: 4 Acknowledged: 2
Undetermined severity findings	0
Informational findings	4 Fixed: 1 Acknowledged: 3

Summary of Findings

The Polymarket V2 codebase implements a full upgrade to Polymarket's prediction market protocol. At a high level, V2 replaces the legacy Conditional Token Framework (CTF) centered design with a modular ERC1155 position system, a unified collateral token, module-specific market logic, operator-mediated order matching, configurable oracle resolution, legacy position migration, and cross-chain bridge support. The architecture is centered around the `PositionManager` contract, which acts as the shared token ledger, while market semantics are delegated to registered modules such as `BinaryModule` and `NegRiskModule`.

The V2 architecture includes the following key features:

- **Position Management:** This replaces the CTF as the primary mechanism for handling user positions. Positions are represented by structured `positionId`s that encode the module, condition/event identity, the event's condition count (arity), condition index, and outcome index directly into the 256-bit token ID. This reduces storage reads/writes and allows routing and authorization to be derived from the ID itself. `PositionManager` acts as the overarching ERC1155 ledger for all positions, while market-specific modules control minting, burning, payout logic, and settlement through checks such as `onlyModuleByPositionId()`.
- **Market Modules:** These modules contain the market-specific logic that was previously spread across CTF-style condition handling. The binary module supports standard YES/NO markets. The neg-risk module supports mutually exclusive multi-condition events, including horizontal split/merge operations and conversion between NO positions and the complementary YES set.
- **Collateral:** A unified collateral token (PMCT / `pUSD`), is introduced backed 1:1 by USDC and USDCe held in an external vault. The onramp and offramp contracts provide permissionless wrapping and unwrapping, while `PermissionedRamp` supports witness-authorized collateral operations. The collateral token contracts have been audited in a previous audit so were considered outside of the scope of this audit.
- **Trading:** The `Exchange` contract provides operator controlled order matching for `PositionManager` positions using EIP-712 signed orders. The exchange supports complementary matches where users trade opposite sides of the same position, as well as same-side matching that mints or merges complementary positions through the relevant module. It also has support for multiple signature types, including EOA wallets, Polymarket proxy wallets, Gnosis Safe-style wallets, and ERC-1271 contract signatures. Fees are charged on filled orders subject to both per-order maximums and a global fee-rate cap.
- **Position Migration:** The migration contracts allow users and operators to migrate legacy CTF and neg-risk positions into the V2 PositionManager system. Migration logic is implemented through module mixins, with binary and neg-risk-specific flows for preparing legacy conditions/events, minting equivalent V2 positions, and resolving migrated positions from legacy oracle state.
- **Oracle:** The `OracleAggregator` provides a modular resolution system where each event can be configured with custom reporter, disputer, and arbitrator modules. The aggregator tracks proposals, dispute windows, arbitration, and finalization, then reports final results to the target market module. The design supports EOA reporters/disputers, quorum-based arbitration, Chainlink-based reporting, and has compatibility with existing UMA Optimistic Oracle flows through `OptimisticOracleModule`.

- **Bridge:** The bridge logic supports cross-chain movement of positions and collateral and allows resolved results to be propagated from the authoritative chain to other deployments. The bridge design separates core bridge accounting and message handling from the concrete transport layer. This audit focused on the core bridge logic within the `BridgeBase` contract; concrete transport integrations such as LayerZero and CCIP were not in the scope of this audit.
- **Routers:** The router contracts provide user-facing entry points for interacting with the system without requiring users or integrators to implement callback logic directly. The routers use the zero-callback pattern by pre-transferring required assets to the module or bridge and then invoking the target operation with `address(0)` as the callback receiver.
- **Auto Redeemer:** The `AutoRedeemer` contract allows a permissioned operator to redeem resolved positions on behalf of users and forward the resulting collateral to them, provided the user has approved the redeemer as an operator for their `PositionManager` position tokens.
- **Upgradeability:** The V2 system introduces upgradeability to several core contracts through UUPS proxies, including `PositionManager`, `CollateralToken`, the core modules, `OracleAggregator`, and `Exchange`.

The audit identified one high severity issue, one medium severity issue as well as several low and informational severity issues. The high severity issue POL-V2-1, allows pUSD drainage by resolving a migrated YES condition through a non-canonical condition ID alias in `resolveMigrationCondition()`, then resolving the canonical conditions in the same event to NO via `resolveConditionToNo()`. The medium-severity issue, POL-V2-2, arises because legacy NegRisk events can finalize with all conditions resolving to NO, which is incompatible with the one-winner invariant assumed by V2 NegRiskModule. If such an event is migrated, including a legacy event that was already resolved to all NO, an attacker can redeem legacy NO positions for collateral, migrate the corresponding YES positions, and then call horizontalMerge() to mint additional unbacked pUSD.We recommend that these issues as well as the low and informational severity issues are addressed thoroughly before deployment and validating each fix with additional tests.

The overall code quality is high. The codebase is modular, well documented, and generally separates shared infrastructure from market-specific logic. On top of this, the coverage of the protocol's test suite was also very good, with the in scope contracts being supported by 688 passing tests and achieving a branch coverage of approximately 88%.

Fix-Review Update 2026-05-18:
During the fix review the client chose to fix or reasonably acknowledge the issues outlined in the audit. Most notably, the high severity issue POL-V2-1 was addressed by making both condition IDs and event IDs user-defined value types (UDVT) with stricter input validations. Additionally, the medium severity issue POL-V2-2 was fixed by preventing migrated legacy NegRisk events from calling `horizontalMerge()` unless the sum of the event's results equals `RESULT_DENOMINATOR`.

The codebase's test suite has been suitably updated to account for the changes made and to validate that the issues were fixed correctly. Overall, the code coverage remains very good.

Fix-Review Update 2026-05-26:
An additional review was conducted, covering two key changes within the protocol:

- The addition of a synthetic "Other" condition within NegRisk events, ensuring one condition per event will resolve to YES.
- The removal all instances of the callback pattern from the entire system.

The effects of both changes were reviewed and no additional issues were found.

ID	DESCRIPTION	SEVERITY	STATUS
POL-V2-1	**Non-Canonical Migration Condition IDs Allow pUSD Drainage**	High	Fixed
POL-V2-2	**Legacy All NO Migration Can Mint Unbacked pUSD**	Medium	Fixed
POL-V2-3	**`OptimisticOracleModule` Cannot Claim OOv2 Deferred Payouts**	Low	Acknowledged
POL-V2-4	**Batch Taker Events Can Overstate Builder Reward Volume**	Low	Fixed
POL-V2-5	**Missing Storage Gaps in Upgradeable Inheritance Chain**	Low	Fixed
POL-V2-6	**Mutable NegRisk Migration Enrollment Can Create Insolvency Risk**	Low	Fixed
POL-V2-7	**Standardize Access Control Patterns**	Low	Acknowledged
POL-V2-8	**Batch Buy Surplus Can Over-Consume Remaining Taker Order Amount**	Low	Fixed
POL-V2-9	**Same-Block Re-Requests Can Revert Due To Duplicate UMA Request IDs**	Informational	Acknowledged

ID	DESCRIPTION	SEVERITY	STATUS
POL-V2-10	**Bridged Markets Not Handled Correctly By External Helper Functions**	Informational	Fixed
POL-V2-11	`validateOrder()` **Does Not Mirror All Order Checks, Producing Inconsistent Off-Chain Validation**	Informational	Acknowledged
POL-V2-12	**Documentation Mismatches Current Implementation Details**	Informational	Acknowledged

Assessment Breakdown

Quantstamp's objective was to evaluate the repository for security-related issues, code quality, and adherence to specification and best practices.

Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
    1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
    2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
    3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
    1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
    2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

The following files were within the scope of the audit:

Files Included

- src/abstract/ERC1155TokenReceiver.sol
- src/auth/InitializableRoles.sol
- src/auth/Roles.sol
- src/bridge/abstract/BridgeBase.sol
- src/bridge/interfaces/IBridge.sol
- src/bridge/interfaces/IBridgeCallback.sol
- src/exchange/Exchange.sol
- src/exchange/OrderStructs.sol

- src/libraries/BridgePayloads.sol
- src/libraries/CrossChainTypes.sol
- src/libraries/ModuleIds.sol
- src/libraries/PositionIdLib.sol
- src/modules/abstract/BaseModule.sol
- src/modules/abstract/ModuleErrors.sol
- src/modules/abstract/OracleModule.sol
- src/modules/BinaryModule.sol
- src/modules/interfaces/IModuleCallback.sol
- src/modules/interfaces/INegRiskModuleCallbacks.sol
- src/modules/migration/BaseMigrationMixin.sol
- src/modules/migration/BinaryMigrationMixin.sol
- src/modules/migration/NegRiskMigrationMixin.sol
- src/modules/NegRiskModule.sol
- src/oracle/abstract/OracleAggregatorErrors.sol
- src/oracle/abstract/OracleAggregatorEvents.sol
- src/oracle/abstract/OracleModuleBase.sol
- src/oracle/interfaces/IArbitratorModule.sol
- src/oracle/interfaces/IBinaryReporter.sol
- src/oracle/interfaces/IDisputerModule.sol
- src/oracle/interfaces/IOptimisticOracleV2.sol
- src/oracle/interfaces/IOptimisticRequester.sol
- src/oracle/interfaces/IOracleAggregator.sol
- src/oracle/interfaces/IReporterModule.sol
- src/oracle/libraries/OptimisticOraclePayoutLib.sol
- src/oracle/mixins/Auth.sol
- src/oracle/mixins/Pausable.sol
- src/oracle/modules/OptimisticOracleModule.sol
- src/oracle/modules/reporters/ChainlinkReporterModule.sol
- src/oracle/OracleAggregator.sol
- src/positionManager/IPositionManagerModule.sol
- src/positionManager/PositionManager.sol
- src/routers/BridgeRouter.sol
- src/routers/CtfRouter.sol
- src/routers/Router.sol
- src/utils/AutoRedeemer.sol

Repo: `https://github.com/Polymarket/polymarket-v2`

Extensions: `sol`

Files Excluded

All other files are out of scope.

Operational Considerations

The following operational assumptions are made of the codebase:

- **The exchange uses an operator-centric trust model:** operators control `matchOrders()` inputs (`conditionId`, `takerFillAmount`, `makerFillAmounts`, fees) and also control preapproval/invalidation flows. Therefore, correct user order execution depends heavily on correct and honest off-chain operator logic. If operator logic is buggy or compromised, outcomes can include value misrouting and stranded inventory, incorrect fill/accounting emissions, and extreme-parameter mis-settlement. In non-COMPLEMENTARY flows, `_mint()` / `_merge()` use the submitted `conditionId` while transfers are keyed by order `tokenId`s, so operator systems must enforce strict `conditionId` to `tokenId` consistency checks. Users should treat this as a high-trust operational assumption, not a low-trust on-chain guarantee.
- **Partial fill precision is trusted to operator execution policy**. `_calculateTakingAmount()` uses floored division when deriving the pro rata taking amount for a fill, so each execution can shift up to one base unit of value to the counterparty whose side benefits from the rounding. If a signed order is filled through many small executions, those discarded remainders can accumulate against the order signer. Because `matchOrders()` fill sizes are operator controlled, operators are expected to enforce sensible minimum fill sizes, batching, and monitoring so repeated tiny fills are not used to concentrate rounding loss against users.
- **Result source coherence assumes a single authoritative oracle path from the hub chain:** if multiple chains can originate result messages for the same logical event, the destination state can diverge unless a single canonical source is enforced. Specifically, because bridged result ingestion bypasses NegRisk aggregate sum validation, inconsistent YES outcomes from different source chains can accumulate on the destination (e.g., chain A reports condition `i` as YES and chain B reports condition `j` as YES, both bridged to chain C), causing `resultsSum` on chain C to exceed `1e6`. **If this condition occurs, unbacked pUSD can be minted**. The client indicated an intent to operate with one oracle on the hub chain, and this assumption was considered throughout the audit.
- **Bridge burn is final on source once message submission succeeds:** `bridgePositions` / `bridgeCollateral` burn assets on the source chain after `_bridgeSend` succeeds, and `BridgeBase` has no native source-side rollback/refund path if destination execution later fails. Delivery is therefore an eventual-success operational assumption that depends on transport-layer recovery procedures (e.g., CCIP manual execution, LayerZero retry/endpoint controls) and correct peer/config management.
- **Manual resolver finalization can set migration-condition outcomes independently from legacy payout numerators:** if a resolver reports a migrated outcome that diverges from the corresponding legacy result, users can settle one side on legacy and the opposite side on

migrated state, creating a cross-system double-settlement path from the same initial collateral basis and introducing insolvency risk. Because resolver-role accounts can finalize migration conditions via `reportResult` outside `resolveMigrationCondition`, settlement consistency for this path relies on operational controls rather than protocol-enforced invariants.

- **Resolver-path finalization can block migration-path legacy settlement:** if a migrated condition is finalized through `reportResult` by a resolver or `OracleAggregator`, `resolveMigrationCondition` for that same condition becomes unavailable. This prevents the normal migration-path legacy redeem-and-settle flow from running for that condition, so operations must ensure equivalent settlement handling is completed through controlled procedures.
- **Repeated OO disputes can leave request rewards in module custody until sweep:** `OptimisticOracleModule.createRequest` pulls reward tokens into the module and `priceDisputed` is only enabled on the first request; re-requests are configured with `callbackOnPriceDisputed = false`. In repeated-dispute paths, reward/refund amounts are therefore not automatically returned to the original request creator by module logic and can remain on module balances until withdrawn. The client indicated this behavior is known and plans to sweep via admin controls, so operations should run periodic reconciliation and sweep procedures for reward tokens.
- **Duplicate markets are possible for the same NegRisk event:** if the legacy event's condition count changes between migration attempts, the same legacy event can map to different V2 markets. This can fragment liquidity across incompatible migrated markets. The first market snapshots only the original `N` conditions; if new conditions are added afterward and the true winner is among those added conditions, the first market can still resolve its original namespace as all-NO, while the second market contains the actual winner.
- **Legacy Other and synthetic Other should be labeled clearly:** a legacy NegRisk event may already include a real condition that represents "Other" to avoid all-NO outcomes. After migration, V2 still has a synthetic Other at `index = arity`, so users and off-chain systems can observe more than one "Other-like" leg. The client must clearly identify which "Other" is the valid settlement leg, because only one "Other" can resolve true.
- **Migrated NegRisk inventory is not identical to native V2 split inventory at migration time:** migration mints migrated condition positions only and does not auto-mint synthetic Other, unlike native V2 horizontal split, which mints one YES per real condition plus synthetic Other.

Key Actors And Their Capabilities

The following authorized roles exist across the various contracts in the codebase:

PositionManager

Owner

- The owner is trusted to handle upgrading the `PositionManager` contract to a new implementation.
- The owner is trusted to grant the `admin` as necessary.

Admin

- The admin is trusted to add and remove valid modules within the system.
- The admin is trusted to add and remove the "cross module authority" of a module within the system.
- The admin is trusted to revoke the `admin` role as necessary.
- The admin has power to grant and revoke the `operator` and `creator` roles as necessary, however they are currently unused within the `PositionManager` contract.

BinaryModule

Owner

- The owner is trusted to handle upgrading the `BinaryModule` contract to a new implementation.
- The owner is trusted to grant the `admin` as necessary.

Creator

- The creator is trusted to handle migrating legacy CTF conditions to V2.

Operator

- The operator can migrate positions on behalf of users if they have approved the `BinaryModule` as spender of their legacy condition tokens.

Bridge

- The bridge is responsible for calling the `mintFromBridge()` and `burnFromBridge()` functions.

Resolver

- Report results for conditions via `reportResult()`.

NegRiskModule

Owner

- The owner is trusted to handle upgrading the `NegRiskModule` contract to a new implementation.
- The owner is trusted to grant the `admin` as necessary.

Creator

- The creator is trusted to handle migrating legacy event IDs to V2.

Operator

- The operator can migration positions on behalf of users if they have approved the `NegRiskModule` as spender of their legacy condition tokens.

Bridge

- The bridge is responsible for calling the `mintFromBridge()` and `burnFromBridge()` functions.

Resolver

- Report results for conditions via `reportResult()`.

Exchange

Owner

- The owner is trusted to handle upgrading the `Exchange` contract to a new implementation.

- The owner is trusted to grant and revoke the `admin` role as necessary.

Admin

- The admin is trusted to grant and revoke the `operator` role as necessary.
- The admin is trusted to pause and unpause trading whenever necessary.
- The admin is trusted to set the number of blocks a user must wait before pausing their own trade eligibility via `setUserPauseBlockInterval()`.

Operator

- The operator is responsible for matching user trade orders via `matchOrders()`.
- The operator can pre-approve a user's order by validating the provided signature via `preapproveOrder()`.
- The operator can also invalidate pre-approved orders.
- The operator can renounce the `operator` role via `renounceOperatorRole()`.

OracleAggregator

Owner

- The owner is trusted to handle upgrading the `OracleAggregator` contract to a new implementation.
- The owner is trusted to grant the `admin` as necessary.

Admin

- The admin is responsible for pausing and unpausing the `OracleAggregator` contract as necessary.
- The admin is trusted to revoke the `admin` role as necessary.
- The admin has power to grant and revoke the `operator` role as necessary.

Operator

- The operator is responsible for creating price requests for a given event via the `initializeRequest()` function.

OptimisticOracleModule

Admin

- The admin is responsible for calling the `forceRerequest()` function in emergency situations where the price did not resolve in a valid manner.
- The admin has the responsibility to set the `OracleAggregator` instance the module is linked to.
- The admin has the power to withdraw any ERC-20 tokens stuck in the `OptimisticOracleModule` contract.

Operator

- The operator is responsible for creating price requests.
- The operator is trusted to give the underlying optimistic oracle contract max approval of any tokens necessary for price reporting to work successfully.

ChainlinkReporterModule

Admin

- The admin is trusted to pause and unpause the `ChainlinkReporterModule` contract as necessary.
- The admin is responsible for setting the correct `priceSource`.
- The admin is responsible for setting the correct `feedId` for a given asset pair.

Bridge

The `owner`, `admin` and `bridge` roles within the bridge logic are expected to be implemented by the concrete transporter contract (such as LZ or CCIP), rather than being implemented within the `BridgeBase` contract.

Owner

- The owner is responsible for setting which modules are supported on a given destination chain via the `setModuleSupported()` function.

Admin

- The admin is responsible for pausing and unpausing the ability to send and receive tokens across the bridge.

AutoRedeemer

Owner

- Add an admin who can manage operators.

Admin

- Add and remove operators as needed.

Operator

- Redeem resolved positions on behalf of users.

Findings

POL-V2-1

Non-Canonical Migration Condition IDs Allow pUSD Drainage

High Fixed

Update

Marked as "Fixed" by the client.

Addressed in: `62a994a0b641b368ffb4d17708876e3cad2d6428`

The client provided the following explanation:

> Non-canonical conditionId aliases now rejected via UDVT ConditionIdLib.from / EventIdLib.from constructors throughout migration, bridge, reportResult, resolveConditionToNo, split/merge paths

File(s) affected: `src/modules/migration/BaseMigrationMixin.sol`, `src/modules/migration/NegRiskMigrationMixin.sol`, `src/modules/NegRiskModule.sol`, `src/bridge/abstract/BridgeBase.sol`, `src/libraries/PositionIdLib.sol`, `src/modules/BinaryModule.sol`, `src/routers/Router.sol`, `src/routers/CtfRouter.sol`

Description: Migrated NegRisk events can be resolved through `resolveMigrationCondition()`, which is permissionless once the corresponding legacy condition has resolved. The function takes a structured V2 `conditionId`, reads the legacy result by deriving the event and condition index from that id, and then stores the V2 result under the raw `conditionId` key.

This creates an identity mismatch. A non-canonical alias such as `canonicalCondition | 1` is a different raw key for V2 result storage, but it still decodes to the same event and condition index for legacy lookup. If the legacy market resolves normally with one YES winner, an attacker can call `resolveMigrationCondition(aliasId)` for that YES condition. This increases `resultsSum[eventId]` to `1_000_000`, while the canonical winning condition remains unresolved in V2.

Once `resultsSum[eventId] == 1_000_000`, anyone can call `resolveConditionToNo()` on the canonical conditions. This lets the canonical V2 event appear all-NO even though the legacy event resolved normally with one YES winner. Because already resolved positions can still be split, a user can repeat the flow by splitting canonical conditions, redeeming all NO positions for principal, and horizontally merging the YES basket for extra pUSD each time. The PoC below demonstrates one iteration ending with `totalInput + amount` after supplying only `totalInput`, but the damage is not capped to that single iteration and can continue draining pUSD until the system is paused.

The impact can extend across chains. Once the non-canonical alias has a stored result on the source chain, `bridgeResult()` can bridge that raw alias result to supported spoke chains because it only checks `module.getResult(_conditionId).length == 2`. On the receiving chain, `_handleResultReceive()` forwards the raw `conditionId` to `reportResult()` from the bridge role. Since bridged results skip the NegRisk sum invariant inside `reportResult()`, the alias can seed the same corrupted event state on the spoke chain. An attacker can then repeat the same `resolveConditionToNo()`, split, redeem, and horizontal merge flow on each affected chain, draining pUSD from multiple deployments before detection and pause.

The source of the regression is that the older implementation resolved through `BaseModule._resolveCondition()`, which called `_requireConditionPrepared()`. Since NegRisk event preparation set an oracle only for each canonical condition index, non-canonical aliases had `oracle[alias] == address(0)` and reverted. The newer migration path no longer enforces this canonical preparation check before storing the result, while legacy lookup still maps the alias back to the same legacy condition.

Exploit Scenario:
Consider the following exploit scenario which demonstrates that demonstrates how a non-canonical `conditionId` can be used to resolve a migrated condition:

```
function test_migrationAlias_allNo_thenSplitRedeemNoAndMergeYes_overExtracts() public {
        uint256 conditionCount = 4;
        uint256 amount = 10_000_000;
        uint256 totalInput = conditionCount * amount;

        _before(conditionCount, amount);

        vm.prank(creator);
        positions.negRiskModule.prepareMigrationEvent(negRiskMarketId);

        bytes32 eventId = _structuredEventId(conditionCount);
        bytes32 canonicalCondition0 = _structuredConditionId(conditionCount, 0);
        bytes32 aliasYes = bytes32(uint256(canonicalCondition0) | 1);

        // Oracle reports outcome for legacy condition on legacy adapter
        vm.prank(oracle);
        legacy.negRiskAdapter.reportOutcome(NegRiskIdLib.getQuestionId(negRiskMarketId, 0), true);

        // Anyone can permissionlessly resolve the migrated coniditon with a non-canonical YES condition
        positions.negRiskModule.resolveMigrationCondition(aliasYes);
        assertEq(positions.negRiskModule.resultsSum(eventId), 1_000_000);

        for (uint256 i; i < conditionCount; ++i) {
            positions.negRiskModule.resolveConditionToNo(_structuredConditionId(conditionCount, i));
        }

        // Check all conditions are NO
        for (uint256 i; i < conditionCount; ++i) {
            uint256[] memory canonicalResult =
                positions.negRiskModule.getResult(_structuredConditionId(conditionCount, i));
            assertEq(canonicalResult[0], 0);
```

```
        assertEq(canonicalResult[1], 1_000_000);
    }

    collateral.usdc.mint(brian, totalInput);

    vm.startPrank(brian);
    collateral.usdc.approve(address(collateral.onramp), totalInput);
    collateral.onramp.wrap(address(collateral.usdc), brian, totalInput);
    collateral.token.approve(address(router), totalInput);
    positions.manager.setApprovalForAll(address(router), true);

    for (uint256 i; i < conditionCount; ++i) {
        router.split(_structuredConditionId(conditionCount, i), amount);
    }

    for (uint256 i; i < conditionCount; ++i) {
        router.redeem(_structuredConditionId(conditionCount, i), 1, amount);
    }

    router.horizontalMerge(eventId, amount);
    vm.stopPrank();

    assertEq(collateral.token.balanceOf(brian), totalInput + amount);
}
```

Recommendation: Reject non-canonical condition IDs before reading or storing migration results by checking the `conditionId` against its decoded version: `require(PositionIdLib.decodeConditionId(uint256(_conditionId)) == _conditionId, InvalidConditionId());`. The proposed check prevents this alias class because `decodeConditionId()` clears the low outcome byte, so aliases such as `canonicalCondition | 1` no longer equal the supplied id.

Apply this guard centrally in `NegRiskModule._validateConditionId()` so `reportResult()`, `resolveConditionToNo()`, and migration finalization (`BaseMigrationMixin.resolveMigrationCondition()`) all reject non-canonical aliases.

Apply the same encoding validation to the other `conditionId` ingress paths as well: `BinaryModule.reportResult()`, `BridgeBase.bridgeResult()`, `BridgeBase._handleResultReceive()`, `Router.split()`, `Router.merge()`, `CtfRouter.splitPosition()`, and `CtfRouter.mergePositions()`. This prevents non-canonical aliases from being accepted through module resolution, bridge result forwarding, or router split/merge flows, and prevents forged bridged payloads from reintroducing raw aliases on spoke chains.

POL-V2-2 Legacy All NO Migration Can Mint Unbacked pUSD

Medium Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `8f51cb46a5a1140d0f94a467e8aa7eca7137061b`, and `07858acf33b49074f579a80aedfbcbc9f41e0f01`
The client implemented the addition of a synthetic "Other" condition within NegRisk events, ensuring one condition per event will resolve to YES.

File(s) affected: `src/modules/migration/BaseMigrationMixin.sol`, `src/modules/migration/NegRiskMigrationMixin.sol`, `src/modules/NegRiskModule.sol`

Description: Legacy NegRisk event resolution can finalize with zero winners, meaning every condition resolves to NO. This differs from the one-winner economic assumption used by the new NegRisk design. If users are able to split positions after such an all NO legacy resolution, they can hold both legacy YES and legacy NO for each condition, redeem the winning NO side for full collateral recovery, and still migrate the corresponding YES side into the new module.

The new module YES basket can then be horizontally merged for additional collateral value. This means a user can recover collateral through the legacy NO redemption path and extract additional value through the new module YES merge path from the same post resolution split set. Even when strict one-winner checks prevent full migration resolution finalization, the collateral extraction path remains reachable before that finalization boundary.

This creates a state where migration safety depends on a legacy oracle invariant that may not hold in practice, rather than being fully enforced by migration time economic constraints. The same attack is also feasible if an already resolved legacy NegRisk event with all NO outcomes is migrated, because migration enrollment does not change the underlying all NO payoff asymmetry between legacy redemption and migrated horizontal merge paths.

Exploit Scenario:

1. A legacy NegRisk event is registered for migration.
2. The legacy event resolves with all conditions finalized as NO.
3. The user holds (or acquires) legacy YES/NO position pairs for each condition, including by splitting collateral after resolution.
4. The user redeems all legacy NO positions and recovers the corresponding collateral.

5. The user migrates the matching legacy YES positions into the new NegRisk module.
6. The user horizontally merges the migrated YES basket and receives additional collateral value.
7. Result: the user extracts more value than the original split input, even if final migration resolution cannot be completed under strict one-winner sum checks.

Recommendation: Consider enforcing one-winner compatibility for migration with one of the following options:

1. Disable `horizontalSplit` and `horizontalMerge` for migrated NegRisk conditions, since migrated legacy positions may follow a different invariant than the native one-winner NegRisk events.
2. Keep horizontal operations enabled, but block `horizontalMerge` for migrated conditions until `resultsSum[eventId] == RESULT_DENOMINATOR`, so merge based collateral extraction is not available before one-winner finalization is established.

POL-V2-3

`OptimisticOracleModule` **Cannot Claim OOv2 Deferred Payouts**

Low Acknowledged

> **Update**
> Marked as "Acknowledged" by the client.

File(s) affected: `src/oracle/modules/OptimisticOracleModule.sol`, `src/oracle/interfaces/IOptimisticOracleV2.sol`

Description: When the OOv2 refunds on dispute to the `OptimisticOracleModule` and the direct ERC20 transfer reverts (for example the module address is blacklisted or the token is paused), `OptimisticOracleV2._transferOrDeferPayout()` falls back to recording the amount in `deferredPayouts[currency][module]` and emits `PayoutDeferred`.

The only way to move those funds out is a call to `OOV2.claimDeferredPayout(currency, repaymentAddress)`, which requires `msg.sender` to equal the deferred recipient (the module itself).

`OptimisticOracleModule` exposes no function that forwards a `claimDeferredPayout()` call to the OOv2, once a deferred payout is recorded against it, the funds remain trapped in the OOv2.

Under normal operation with standard bond tokens the transfer to the module is expected to succeed, so the defer path is unlikely to be triggered.

Recommendation: Add an access-controlled function (similar to `withdrawERC20()`) to `OptimisticOracleModule` that allows claiming deferred payouts via `claimDeferredPayout()` on OOv2 and sends the recovered funds to a specified address..

POL-V2-4 Batch Taker Events Can Overstate Builder Reward Volume

Low Fixed

> **Update**
> Marked as "Fixed" by the client.
> Addressed in: `f497e63a05d19ee7c046bd526275b276deb865e1`

File(s) affected: `src/exchange/Exchange.sol`

Description: Batch matches can settle a taker order for less than the gross amount reported in the taker `OrderFilled` and `OrdersMatched` events. In the batch path, `_matchOrders()` sets `takerMakingAmount` to `takerFillAmount` before settlement completes, then `_emitTakerEvents()` emits that gross amount even if the batch settlement refunds collateral or positions back to the taker.

If builder rewards or volume accounting are calculated from taker `OrderFilled` events keyed by `builder`, downstream systems can attribute rewards using the gross requested fill instead of the net settled amount.

Exploit Scenario:

1. A builder creates a taker buy order that is willing to spend 70 collateral to receive 100 YES.
2. The batch match crosses it with maker liquidity that only requires 50 collateral after minting and netting.
3. Settlement refunds 20 collateral to the taker, so the taker actually settles 50 collateral of volume.
4. The taker `OrderFilled` event still reports 70 as the filled amount for the builder attributed order.
5. A rewards indexer consumes that event and emits or credits builder rewards as if 70 collateral settled, causing the reward amount to be higher than intended.

Recommendation: Emit taker batch match accounting from the net settled amount after refunds rather than from `ctx.takerFillAmount`.

A possible fix is to have `_executeBatchBuyMatch()` and `_executeBatchSellMatch()` return both the amount used to update taker order fill state and the actual net settled amount after refunds, then pass the net amount into `_emitTakerEvents()` for taker `OrderFilled` and `OrdersMatched` emission.

Add event tests where batch buy and batch sell matches produce refunds and assert that taker event values equal the net settled amount.

POL-V2-5 Missing Storage Gaps in Upgradeable Inheritance Chain

Low Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `549ae7cdaa9248817bfd70c51ea1d4977b1bc504`

File(s) affected: `src/modules/migration/NegRiskMigrationMixin.sol`, `src/modules/migration/BaseMigrationMixin.sol`, `src/modules/abstract/OracleModule.sol`, `src/modules/abstract/BaseModule.sol`, `src/oracle/mixins/Pausable.sol`

Description: Several upgradeable contracts inherit from stateful base contracts/mixins that do not reserve storage space for future upgrades. This is especially relevant for `NegRiskModule`, whose storage layout places inherited state before state declared directly in `NegRiskModule`.

If a future upgrade adds a state variable to `NegRiskMigrationMixin`, it will be inserted before the `NegRiskModule` contract's own storage variables. This would shift the existing storage layout and corrupt the expected values stored in these slots.

The same upgrade-safety concern applies to other stateful inherited contracts used by upgradeable implementations, including:

- `BaseModule.sol`
- `BaseMigrationMixin.sol`
- `BinaryMigrationMixin.sol`
- `OracleModule.sol`
- `oracle/mixins/Pausable.sol`

These contracts would also benefit from reserved storage space or an alternative storage isolation pattern.

Recommendation: Adopt one of the following approaches for the affected upgradeable inheritance chains:

- Add storage gaps to stateful inherited contracts, and reduce the gap size whenever new variables are added in future upgrades.
- Alternatively, use a namespaced storage layout, such as ERC-7201-style storage namespaces, so future variables can be added without shifting the linearized storage layout.

POL-V2-6
Mutable NegRisk Migration Enrollment Can Create Insolvency Risk

Low | Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `4eb4e0c23b14bbb21f8047c177be15ae2e360427`

File(s) affected: `src/modules/migration/NegRiskMigrationMixin.sol`, `src/modules/migration/BaseMigrationMixin.sol`, `src/modules/NegRiskModule.sol`, `src/libraries/PositionIdLib.sol`

Description: `prepareMigrationEvent()` binds migration state to a structured `eventId` that includes the current legacy condition count (encoded as arity), then snapshots legacy condition index mappings for that count. The same legacy event can later be enrolled again under a different structured `eventId` if the legacy condition count changes, because uniqueness is enforced per structured `eventId` key rather than per legacy event ID.

This creates a moving migration target for a single legacy market: previously-migrated positions continue to exist under the earlier condition-count namespace, while future migrations of the same legacy conditions can be redirected into a new condition-count namespace. If the legacy layout changes after initial enrollment (for example, additional conditions are appended), migrated accounting and resolution assumptions can diverge from the original snapshot, and the protocol can end up operating multiple incompatible migrated views of one legacy event.

Under specific outcome patterns, this divergence can produce migrated states that conflict with one-winner assumptions used for pUSD safety and can reintroduce the same collateral extraction risk class as all NO style migration outcomes.

Exploit Scenario:

1. A legacy NegRisk event is enrolled for migration when condition count is `N` (encoded as arity).
2. Migration begins and users receive migrated positions keyed to the `N`-condition event namespace.
3. Later, due to operational error, the legacy event is extended to condition count `N + k`, and migration enrollment is executed again.
4. New migrations for the same legacy event now map into the new `N + k` namespace, while old migrated positions remain in the prior namespace.
5. Resolution and payout handling proceed across inconsistent migrated namespaces for one underlying legacy market.
6. The eventual winning legacy condition index lands in the appended range, i.e. `winnerIndex` is in `[N, N + k − 1]`.
7. In the earlier `N`-condition namespace, every condition is now effectively NO-winning (all-NO from that namespace perspective).
8. After re-enrollment, additional migrations for those legacy conditions route to the new `N + k` namespace, but any YES positions that were already migrated into the earlier `N` namespace remain valid there.
9. An attacker holding that earlier-namespace YES basket (for example from pre-change migration) can redeem legacy NO on the original `N` conditions and then call `horizontalMerge` on the earlier YES basket to mint pUSD.
10. The attacker extracts extra pUSD-equivalent value from the earlier namespace, creating insolvency risk.

Recommendation: Enforce one-time enrollment per legacy event ID, not per structured `eventId` derived from condition count (arity).

POL-V2-7 Standardize Access Control Patterns

Low | Acknowledged

> **Update**
> Marked as "Acknowledged" by the client.
> The client provided the following explanation:
>
> > Too much blast radius, large change for little gain, redundant selectors are irrelevant.

File(s) affected: `src/*`

Description: The contracts currently use a mixture of access control abstractions, including `Auth`, `Roles`, `InitializeableRoles`, and direct inheritance from Solady's `OwnableRoles`.

This inconsistency makes it harder to determine which roles exist on each contract, what permissions they grant, and which administrative functions are actually relevant. In some cases, contracts expose role management functions for roles that are not used by that contract, increasing the administrative surface area without a corresponding need.

Recommendation: Consider standardizing access control across the codebase around a consistent set of abstractions and role definitions. Contracts should only expose role management functionality for roles they actually use, and inherited access control modules should be limited to the permissions required by each contract.

POL-V2-8

Batch Buy Surplus Can Over-Consume Remaining Taker Order Amount

Low | Fixed

> **Update**
> Marked as "Fixed" by the client.
> Addressed in: `8009ad916cbacb4e951f71e45e4528ca2466f0a9`

File(s) affected: `src/exchange/Exchange.sol`

Description: Batch order matching updates the taker order status in the `_matchBatchOrders()` function with the gross `takerFillAmount` before settlement completes. For batch buy matches that produce a collateral refund, `_executeBatchBuyMatch()` later computes the net `takerMakingAmount` as `takerFillAmount - collateralRefund`, but the stored order remaining value has already been reduced by the gross amount. The remaining calculation consumes a taker order faster than the net settled amount, potentially marking an order as filled even though only part of the signed economic size was executed.

Exploit Scenario:

1. Alice signs a BUY order offering 90 pUSD for at least 100 YES.
2. The operator submits a batch buy match with `takerFillAmount = 70`.
3. The batch includes a NO side maker that contributes 35 pUSD, allowing the exchange to mint 100 YES and 100 NO.
4. Alice receives 100 YES.
5. The actual net cost to Alice is 65 pUSD, because 5 pUSD is refunded after settlement.
6. However, `_validateAndUpdate()` already reduced Alice order remaining by 70, not by the actual 65.
7. Alice remaining becomes 20 pUSD, while it should be 25 pUSD.
8. Repeating this pattern across several partial fills can close Alice order earlier than the net amount actually spent.

Recommendation: Validate the taker order before batch settlement, then update its status after settlement using the actual settled maker leg.

POL-V2-9

Same-Block Re-Requests Can Revert Due To Duplicate UMA Request IDs

Informational | Acknowledged

> **Update**
> Marked as "Acknowledged" by the client.

File(s) affected: `src/oracle/modules/OptimisticOracleModule.sol`

Description: `OptimisticOracleModule` creates UMA requests using a request tuple that includes the module address, identifier, timestamp, and ancillary data. When `_reRequest()` is called after a dispute or through `forceRerequest()`, it reuses the same identifier and ancillary data, and relies on `block.timestamp` as the only value that changes the underlying UMA request ID.

If two re-requests for the same condition are attempted in the same block with the same ancillary data, the second call attempts to create an already-existing UMA request. The underlying Optimistic Oracle rejects duplicate request IDs, causing the later request to revert.

This creates a temporary griefing vector against admin `forceRerequest()` calls. A user can front-run an admin's `forceRerequest()` by disputing the current price request. The dispute callback triggers `_reRequest()` first, creating a new request with the existing ancillary data and current block timestamp. If the admin transaction then executes in the same block with the same ancillary data, it reverts before applying the admin-selected bond token, bond amount, liveness, or zero-reward reset.

The impact is temporary liveness degradation rather than permanent request blocking. The admin can retry in a later block, or use different ancillary data, but the failed force action may delay recovery and leave the condition on the automatically created re-request using the prior request parameters.

Recommendation: Ensure that each re-request has a unique UMA request tuple independent of `block.timestamp`. For example, if changing `ancillaryData` between requests for the same `conditionId` is acceptable, maintain a per condition round or nonce and include it in the ancillary data used for each request.

POL-V2-10
Bridged Markets Not Handled Correctly By External Helper Functions

Informational · Fixed

Update

Marked as "Fixed" by the client.
Addressed in: `ca67f749a1d51ad95779b0f789d4577a35a9504f`
The client provided the following explanation:

> `chainid` removed from positionId encoding entirely

File(s) affected: `src/modules/BinaryModule.sol`, `src/modules/NegRiskModule.sol`

Description: Structured position IDs derive their `conditionId` / `eventId` from a `baseHash`, and the `baseHash` includes the condition's original chain ID. The functions `NegRiskModule.getEventId()` and `BinaryModule.getConditionId()` hard-code the `_chainId` component of the computed `conditionId` to be `block.chainid`. For conditions that originate on one chain and are later bridged to another, the bridged positions retain their original source-chain IDs. Calling these helpers on the destination chain with the same market data will instead compute a different destination-chain ID. As a result, external integrations, indexers, frontends, or other offchain systems relying on these helpers may display incorrect market IDs, query incorrect balances/results, or build transactions against the wrong condition/event.

Recommendation: Consider allowing the caller to specify a `_chainId` to ensure the correct condition ID can be derived from these functions.

POL-V2-11
`validateOrder()` Does Not Mirror All Order Checks, Producing Inconsistent Off-Chain Validation

Informational · Acknowledged

Update

Marked as "Acknowledged" by the client.

File(s) affected: `src/exchange/Exchange.sol`

Description: `validateOrder()` only validates order level state and signature data. It does not take execution specific context such as `fillAmount`, `fee`, `conditionId`, or maker side orders, so it cannot mirror the checks performed during `matchOrders()`. As a result, an order can pass `validateOrder()` but still revert during execution due to remaining capacity, fee bounds, token ID alignment, or other match specific checks.

Recommendation: Rename `validateOrder()` to something that reflects its limited scope, such as `validateRawOrder()` or `preflightOrderCheck()`, and document that passing it does not guarantee a successful match. If execution level validation is needed, add a separate helper that accepts the relevant match context.

POL-V2-12
Documentation Mismatches Current Implementation Details

Informational · Acknowledged

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

> We will take a closer look at the documentation.

File(s) affected: `src/oracle/modules/README.md`, `src/oracle/modules/OptimisticOracleModule.sol`, `src/oracle/OracleAggregator.sol`, `docs/architecture.md`, `docs/position-ids.md`, `docs/position-manager.md`, `docs/modules.md`, `docs/exchange.md`, `docs/router.md`, `docs/oracle.md`, `docs/auth.md`, `docs/bridge.md`, `docs/collateral.md`, `docs/adapters.md`, `docs/migration.md`

Description: The documentation contains stale information compared to the actual audited implementation, likely because the code changed shortly before the audit happened without updating the docs.

The verified misalignments are:

1. `src/oracle/modules/README.md` states that `onArbitrationTriggered()` reverts with `ArbitrationNotExpected` as a safety check. `OptimisticOracleModule` does not implement `onArbitrationTriggered()` and does not define `ArbitrationNotExpected`. If `OracleAggregator.disputeResult()` reaches arbitration for a request configured with `OptimisticOracleModule` as the arbitrator, the call is made through `IArbitratorModule(cfg.arbitratorModule).onArbitrationTriggered(...)`. Because the function is not implemented and there is no fallback handler, the call reverts with a generic error instead of the documented custom error.
2. `docs/position-ids.md` and `docs/position-manager.md` document `PositionManager.getPositionId(conditionId, outcomeIndex)`, but that function does not exist in `PositionManager`.
3. `docs/position-ids.md` references router callback helpers that do not exist in `Router`.
4. `docs/architecture.md` and `docs/position-manager.md` use stale PMCT and `collateralToken` naming instead of current `pUSD` and `COLLATERAL_TOKEN` naming.
5. `docs/exchange.md` documents stale order fields, EIP712 domain values, initializer, constructor, fee controls, pause controls, `matchOrders` argument order, and view names.
6. `docs/modules.md` describes an old oracle assignment model. Current modules use resolver and bridge roles.
7. `docs/modules.md` lists stale bridge and OracleModule functions, and an incomplete `convert` signature.
8. `docs/router.md` has stale merge behavior, AutoRedeemer API, and omits BridgeRouter refund forwarding.
9. `docs/oracle.md` has stale auth details, and has the wrong initializer signature.
10. `docs/auth.md` omits resolver role support, overstates `InitializableRoles`, and repeats stale Exchange and AutoRedeemer APIs.
11. `docs/bridge.md` documents module support by module id, stale message types, stale payload helpers, and incorrect duplicate result behavior.
12. `docs/migration.md` documents stale migration argument shapes, stale operator overloads, overbroad revert behavior, and internal helpers as public API.

Recommendation: Fix every misalignment listed in the description so the documentation matches the current implementation.

Auditor Suggestions

S1 Improve Validations

Acknowledged

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

> Seems to have no prod impact, we will save the gas.

File(s) affected: `src/bridge/abstract/BridgeBase.sol`, `src/exchange/Exchange.sol`, `src/modules/migration/BaseMigrationMixin.sol`, `src/modules/migration/NegRiskMigrationMixin.sol`, `src/oracle/OracleAggregator.sol`

Description: Consider adding input validations so invalid arguments fail with clear errors. The following is a non-exhaustive list of suggested validations:

1. In `BridgeBase.bridgePositions()`, require each bridged position amount to be non zero so zero amount position entries cannot be sent.
2. In `BridgeBase.bridgeCollateral()`, require `_amount > 0` so zero amount collateral messages cannot be sent.
3. In `BridgeBase._processMessage()`, explicitly revert on unsupported `MessageType` values instead of silently doing no op behavior.
4. In `BridgeBase._handlePositionsReceive()`, require that the decoded bridged positions array is non empty before accessing index `0`, and revert with a dedicated error when empty.
5. In `BridgeBase._handlePositionsReceive()` and `BridgeBase._handleCollateralReceive()`, require decoded `recipient != address(0)` before minting. Mint will revert in `ERC1155.sol` with `TransferToZeroAddress()` when the recipient is `address(0)`, consider adding a dedicated error.
6. In `BridgeBase.pauseSend()`, `BridgeBase.unpauseSend()`, `BridgeBase.pauseReceive()`, and `BridgeBase.unpauseReceive()`, require a real state transition before emitting events (for example, revert if already paused/unpaused).
7. In `Exchange` order validation, require `order.makerAmount <= type(uint248).max` before status packing so remaining amount storage cannot silently truncate.

8. In `BinaryMigrationMixin.prepareMigrationCondition()`, require `_legacyConditionId != bytes32(0)`.
9. In `BaseMigrationMixin.resolveMigrationCondition()`, require `_getLegacyConditionIdForResolve(_conditionId) != bytes32(0)` before reading payout numerators and revert with `MigrationNotRegistered()` when missing.
10. In `OracleAggregator.initializeRequest()`, require `reporterThreshold <= reporterModules.length` and `disputerThreshold <= disputerModules.length`.
11. In `OracleAggregator.initializeRequest()`, require `reporterModules.length > 0` so reporter thresholds remain reachable.
12. In `OracleAggregator.initializeRequest()`, require `disputerThreshold == 0` when `disputerModules.length == 0` to avoid inconsistent no disputer configurations.
13. In `OracleAggregator._registerReporterModules()` and `_registerDisputerModules()`, require `config.module != address(0)` and reject duplicate module entries.

Recommendation: Consider implementing the listed validation checks.

S2 Miscellaneous Improvements

Fixed

> **Update**
> Marked as "Fixed" by the client.
> Addressed in: `dcb4863296ed1061d1899c0e74aca18bd553bce8`

File(s) affected: `src/oracle/OracleAggregator.sol`, `src/modules/migration/BinaryMigrationMixin.sol`, `src/modules/migration/BaseMigrationMixin.sol`, `src/libraries/PositionIdLib.sol`

Description: The following changes would improve the overall quality and consistency of the codebase:

1. In `OracleAggregator._finalizeCondition()`, move the `binaryResult` allocation outside the loop when the same buffer can be reused safely.
2. In `BinaryMigrationMixin.prepareMigrationCondition()`, emit an event when a legacy binary migration condition is prepared, matching the NegRisk migration flow.
3. In `OracleAggregator._registerReporterModules()` and `_registerDisputerModules()`, use `++i` for style consistency and minor gas savings.
4. In `BaseMigrationMixin._buildTransferAndMint()` and `_mergeComplementaryAndEmit()`, use `++i` for style consistency and minor gas savings.
5. In `PositionIdLib`, remove the redundant `_moduleId & MODULE_MASK` before shifting into the top byte: bits above the low 8 are discarded by the `<< MODULE_SHIFT` operation, unlike lower fields where masks prevent spillover into neighboring slots.

Recommendation: Consider implementing the suggested changes.

S3 Unset Module Matches Can Revert Silently

Acknowledged

> **Update**
> Marked as "Acknowledged" by the client.
> The client provided the following explanation:
>
> > Opting to keep the gas lean.

File(s) affected: `src/exchange/Exchange.sol`

Description: `matchOrders()` initializes the batch execution context with an unset module address and resolves the module only when execution takes the non-complementary batch branch. If the module lookup returns a zero address because the module is unregistered or misconfigured, execution can still continue into batch settlement before failing.

When settlement reaches module-level split or merge operations with a zero target, the call reverts with empty revert data. The state is still safe because the transaction reverts atomically, but the failure is operationally opaque: monitoring systems and operators receive a generic failure with no structured reason, which slows diagnosis and incident response.

Recommendation: Add an early validation after module resolution in the batch path and revert with a dedicated custom error when the resolved module is unset.

Definitions

- **High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- **Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.

- **Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- **Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- **Undetermined** – The impact of the issue is uncertain.
- **Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- **Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- **Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Appendix

File Signatures

The following are the SHA-256 hashes of the reviewed files. A file with a different SHA-256 hash has been modified, intentionally or otherwise, after the security review. You are cautioned that a different SHA-256 hash could be (but is not necessarily) an indication of a changed condition or potential vulnerability that was not within the scope of the review.

Files

Repo: `https://github.com/Polymarket/polymarket-v2`

- `ffc...724 ./src/abstract/ERC1155TokenReceiver.sol`
- `73f...cf1 ./src/auth/InitializableRoles.sol`
- `7c7...9af ./src/auth/Roles.sol`
- `044...0d2 ./src/bridge/abstract/BridgeBase.sol`
- `7f1...d37 ./src/bridge/interfaces/IBridge.sol`
- `acb...c2b ./src/bridge/interfaces/IBridgeCallback.sol`
- `ab0...2af ./src/collateral/CollateralOfframp.sol`
- `98b...4af ./src/collateral/CollateralOnramp.sol`
- `03c...07e ./src/collateral/CollateralToken.sol`
- `5c0...cfa ./src/collateral/PermissionedRamp.sol`
- `140...8d6 ./src/collateral/abstract/CollateralErrors.sol`
- `9f9...076 ./src/collateral/abstract/Pausable.sol`
- `e00...075 ./src/collateral/dev/CollateralSetup.sol`
- `a19...02f ./src/collateral/dev/CollateralSetup.t.sol`
- `d07...df7 ./src/collateral/interfaces/ICollateralToken.sol`
- `d04...9b8 ./src/collateral/interfaces/ICollateralTokenCallbacks.sol`
- `49f...947 ./src/exchange/Exchange.sol`
- `ef2...46b ./src/exchange/OrderStructs.sol`
- `8b8...b53 ./src/libraries/BridgePayloads.sol`
- `a63...1b7 ./src/libraries/CrossChainTypes.sol`
- `241...ee0 ./src/libraries/ModuleIds.sol`
- `b4d...b40 ./src/libraries/PositionIdLib.sol`
- `a5d...929 ./src/modules/BinaryModule.sol`
- `f4b...91b ./src/modules/NegRiskModule.sol`
- `152...ed8 ./src/modules/abstract/BaseModule.sol`
- `524...47b ./src/modules/abstract/ModuleErrors.sol`
- `710...d2d ./src/modules/abstract/OracleModule.sol`
- `1b9...6b3 ./src/modules/interfaces/IModuleCallback.sol`
- `71a...925 ./src/modules/interfaces/INegRiskModuleCallbacks.sol`
- `ecd...2d0 ./src/modules/migration/BaseMigrationMixin.sol`
- `d73...0c0 ./src/modules/migration/BinaryMigrationMixin.sol`
- `b2f...e90 ./src/modules/migration/NegRiskMigrationMixin.sol`
- `db7...812 ./src/oracle/OracleAggregator.sol`
- `c13...7c7 ./src/oracle/abstract/OracleAggregatorErrors.sol`
- `332...0c6 ./src/oracle/abstract/OracleAggregatorEvents.sol`

- d0f...6f4 ./src/oracle/abstract/OracleModuleBase.sol
- 89d...157 ./src/oracle/interfaces/IArbitratorModule.sol
- e06...d4d ./src/oracle/interfaces/IBinaryReporter.sol
- 213...679 ./src/oracle/interfaces/IDisputerModule.sol
- 415...226 ./src/oracle/interfaces/IOptimisticOracleV2.sol
- abe...622 ./src/oracle/interfaces/IOptimisticRequester.sol
- e90...0c5 ./src/oracle/interfaces/IOracleAggregator.sol
- 4c6...9e8 ./src/oracle/interfaces/IReporterModule.sol
- e82...3aa ./src/oracle/libraries/OptimisticOraclePayoutLib.sol
- d97...89d ./src/oracle/mixins/Auth.sol
- 043...c43 ./src/oracle/mixins/Pausable.sol
- c62...ead ./src/oracle/modules/OptimisticOracleModule.sol
- e83...58e ./src/oracle/modules/arbitrators/QuorumArbitratorModule.sol
- 45d...5a0 ./src/oracle/modules/reporters/ChainlinkReporterModule.sol
- 7f1...0e1 ./src/positionManager/IPositionManagerModule.sol
- 065...023 ./src/positionManager/PositionManager.sol
- 6f3...4f7 ./src/routers/BridgeRouter.sol
- 2da...48b ./src/routers/CtfRouter.sol
- 423...72c ./src/routers/Router.sol
- e8d...eb9 ./src/utils/AutoRedeemer.sol

Test Suite Results

The test suite can be run with the following command:

```
forge build
forge test --no-match-contract "LzBridge|CcipBridge|EOA|Quorum|Adapter"
```

```
Ran 5 tests for src/modules/test/BinaryMigration.t.sol:BinaryMigrationTest_operatorBatchMigrate
[PASS] test_batchMigratePositions() (gas: 551562)
[PASS] test_revert_amountsLengthMismatch() (gas: 94341)
[PASS] test_revert_invalidArrayLength() (gas: 94441)
[PASS] test_revert_positionIdsLengthMismatch() (gas: 94297)
[PASS] test_revert_unauthorized() (gas: 92043)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 2.29ms (651.50µs CPU time)

Ran 2 tests for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_hasResult
[PASS] test_hasResult_false() (gas: 37404)
[PASS] test_hasResult_true() (gas: 101759)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.37ms (48.04µs CPU time)

Ran 2 tests for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_hasResult
[PASS] test_hasResult_false() (gas: 37568)
[PASS] test_hasResult_true() (gas: 200753)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.43ms (62.88µs CPU time)

Ran 8 tests for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_bridge
[PASS] test_burnFromBridge() (gas: 97586)
[PASS] test_burnFromBridge_batch() (gas: 147963)
[PASS] test_mintFromBridge() (gas: 86400)
[PASS] test_mintFromBridge_thenRedeem() (gas: 240924)
[PASS] test_mintFromBridge_zeroAmount() (gas: 58496)
[PASS] test_onERC1155BatchReceived_normalTransfer() (gas: 138008)
[PASS] test_onERC1155Received_normalTransfer() (gas: 96959)
[PASS] test_revert_mintFromBridge_unauthorized() (gas: 42826)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 2.74ms (347.25µs CPU time)

Ran 3 tests for src/utils/test/AutoRedeemer.t.sol:AutoRedeemerTest_deployment
[PASS] test_deployment() (gas: 60250)
[PASS] test_revert_cannotReinitialize() (gas: 22622)
```

```
[PASS] test_revert_implementationInitializerDisabled() (gas: 2924491)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 2.75ms (128.92µs CPU time)

Ran 4 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_mergeOnCondition
[PASS] test_mergeOnCondition() (gas: 537931)
[PASS] test_revert_mergeOnCondition_parentMustBeYes() (gas: 197526)
[PASS] test_revert_mergeOnCondition_parentUnprepared() (gas: 143072)
[PASS] test_revert_mergeOnCondition_referenceMustBeConditionId() (gas: 6291)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 2.78ms (281.38µs CPU time)

Ran 2 tests for
src/oracle/test/modules/reporters/ChainlinkReporterModule.t.sol:ChainlinkReporterModuleTest_initializer
[PASS] test_revert_initialize_onImplementation() (gas: 1554143)
[PASS] test_revert_initialize_twice() (gas: 24969)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.81ms (59.12µs CPU time)

Ran 1 test for src/exchange/test/MatchOrders.t.sol:MatchOrdersTest_complementaryNotCrossing
[PASS] test_revert_complementaryNotCrossing() (gas: 441252)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 3.14ms (406.83µs CPU time)

Ran 5 tests for src/oracle/test/OracleAggregator.t.sol:OracleAggregatorTest_pausable
[PASS] test_revert_disputeResult_whenPaused() (gas: 600321)
[PASS] test_revert_finalize_whenPaused() (gas: 598350)
[PASS] test_revert_reportResult_whenPaused() (gas: 443838)
[PASS] test_revert_resolveResult_whenPaused() (gas: 417145)
[PASS] test_unpauseOracle() (gas: 33812)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 3.58ms (415.29µs CPU time)

Ran 3 tests for src/modules/test/BinaryMigration.t.sol:BinaryMigrationTest_prepareCondition
[PASS] test_revert_alreadyPrepared() (gas: 87155)
[PASS] test_revert_questionNotPrepared() (gas: 25168)
[PASS] test_revert_wrongOutcomeCount() (gas: 64316)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 1.42ms (53.46µs CPU time)

Ran 1 test for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_initialize
[PASS] test_revert_cannotReinitialize() (gas: 22678)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.96ms (11.38µs CPU time)

Ran 2 tests for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_horizontalMerge
[PASS] test_horizontalMerge() (gas: 508514)
[PASS] test_revert_invalidEventId() (gas: 16661)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.15ms (134.42µs CPU time)

Ran 4 tests for src/modules/test/BinaryMigration.t.sol:BinaryMigrationTest_resolveMigrationCondition
[PASS] test_resolveMigrationCondition() (gas: 450781)
[PASS] test_revert_notResolvedOnLegacy() (gas: 93526)
[PASS] test_revert_resolutionPaused() (gas: 307636)
[PASS] test_unpauseAllowsResolution() (gas: 476757)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 4.80ms (2.28ms CPU time)

Ran 6 tests for src/auth/test/Roles.t.sol:RolesTest_admin
[PASS] test_addAdmin() (gas: 37608)
[PASS] test_adminOnlyFunction() (gas: 12715)
[PASS] test_removeAdmin() (gas: 32776)
[PASS] test_revert_addAdmin_unauthorized() (gas: 12921)
[PASS] test_revert_adminOnlyFunction_unauthorized() (gas: 12895)
[PASS] test_revert_removeAdmin_unauthorized() (gas: 42265)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 401.00µs (63.17µs CPU time)

Ran 6 tests for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_direct
[PASS] test_merge_direct() (gas: 363600)
[PASS] test_redeem_direct() (gas: 392671)
[PASS] test_revert_merge_noPreTransfer() (gas: 86541)
[PASS] test_revert_redeem_noPreTransfer() (gas: 151206)
[PASS] test_revert_split_noPreTransfer() (gas: 92280)
[PASS] test_split_direct() (gas: 225741)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 2.25ms (298.92µs CPU time)

Ran 1 test for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_split
[PASS] test_split() (gas: 279726)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 5.16ms (67.71µs CPU time)
```

```
Ran 6 tests for src/auth/test/Roles.t.sol:RolesTest_bridge
[PASS] test_addBridge() (gas: 37715)
[PASS] test_bridgeOnlyFunction() (gas: 12646)
[PASS] test_removeBridge() (gas: 19251)
[PASS] test_revert_addBridge_unauthorized() (gas: 13024)
[PASS] test_revert_bridgeOnlyFunction_unauthorized() (gas: 12893)
[PASS] test_revert_removeBridge_unauthorized() (gas: 15120)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 381.46µs (51.96µs CPU time)

Ran 3 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_multicall
[PASS] test_multicall_canReturnCollateralFromRelatedNoPositions() (gas: 743387)
[PASS] test_multicall_twoSplits() (gas: 342923)
[PASS] test_revert_multicall_bubblesInnerError() (gas: 18245)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 2.53ms (379.92µs CPU time)

Ran 8 tests for
src/oracle/test/modules/reporters/ChainlinkReporterModule.t.sol:ChainlinkReporterModuleTest_report
[PASS] test_priceDown() (gas: 529685)
[PASS] test_priceUp() (gas: 600913)
[PASS] test_revert_eventNotInitialized() (gas: 17549)
[PASS] test_revert_feedIdNotSet() (gas: 321573)
[PASS] test_revert_priceMissingInDataStore() (gas: 380457)
[PASS] test_revert_report_nonBinaryRequestId() (gas: 318849)
[PASS] test_revert_whenPaused() (gas: 443193)
[PASS] test_succeedsBeforeWindowEndWhenPricesExist() (gas: 529113)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 2.70ms (560.50µs CPU time)

Ran 1 test for src/oracle/test/OracleAggregator.t.sol:OracleAggregatorTest_report
[PASS] test_revert_invalidResultLength() (gas: 442921)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.09ms (86.38µs CPU time)

Ran 6 tests for src/auth/test/Roles.t.sol:RolesTest_creator
[PASS] test_addCreator() (gas: 37692)
[PASS] test_creatorOnlyFunction() (gas: 12644)
[PASS] test_removeCreator() (gas: 19365)
[PASS] test_revert_addCreator_unauthorized() (gas: 13012)
[PASS] test_revert_creatorOnlyFunction_unauthorized() (gas: 12892)
[PASS] test_revert_removeCreator_unauthorized() (gas: 15131)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 375.96µs (47.50µs CPU time)

Ran 7 tests for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_resolver
[PASS] test_addResolver_thenReport() (gas: 106731)
[PASS] test_pauseResolution() (gas: 44700)
[PASS] test_pauseResolver() (gas: 44765)
[PASS] test_revert_reportResult_resolutionPaused() (gas: 52730)
[PASS] test_revert_reportResult_resolverPaused() (gas: 50561)
[PASS] test_unpauseResolution() (gas: 35067)
[PASS] test_unpauseResolver() (gas: 35345)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 2.07ms (138.79µs CPU time)

Ran 6 tests for src/auth/test/Roles.t.sol:RolesTest_operator
[PASS] test_addOperator() (gas: 37759)
[PASS] test_operatorOnlyFunction() (gas: 12735)
[PASS] test_removeOperator() (gas: 19270)
[PASS] test_revert_addOperator_unauthorized() (gas: 13035)
[PASS] test_revert_operatorOnlyFunction_unauthorized() (gas: 12895)
[PASS] test_revert_removeOperator_unauthorized() (gas: 15109)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 378.25µs (52.58µs CPU time)

Ran 2 tests for src/exchange/test/MatchOrders.t.sol:MatchOrdersTest_complementarySellNotCrossing
[PASS] test_revert_complementarySellNotCrossing_withFees() (gas: 458030)
[PASS] test_revert_complementarySellNotCrossing_zeroFee() (gas: 458068)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.89ms (582.17µs CPU time)

Ran 1 test for src/auth/test/Roles.t.sol:RolesTest_resolver
[PASS] test_addRemoveResolver() (gas: 32024)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 342.83µs (19.42µs CPU time)

Ran 2 tests for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_horizontalSplit
[PASS] test_horizontalSplit() (gas: 340500)
[PASS] test_revert_invalidEventId() (gas: 222554)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.00ms (122.13µs CPU time)
```

```
Ran 4 tests for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_getPayout
[PASS] test_fullPayout() (gas: 104878)
[PASS] test_partialPayout() (gas: 124854)
[PASS] test_revert_conditionNotResolved() (gas: 40360)
[PASS] test_revert_invalidOutcomeIndex() (gas: 101817)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 2.37ms (106.25µs CPU time)

Ran 3 tests for src/utils/test/AutoRedeemer.t.sol:AutoRedeemerTest_events
[PASS] test_redeemEmitsPayout() (gas: 448817)
[PASS] test_redeemLegacyBinaryEmitsPayout() (gas: 466482)
[PASS] test_redeemLegacyNegRiskEmitsPayout() (gas: 814254)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 4.72ms (2.50ms CPU time)

Ran 4 tests for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_upgrade
[PASS] test_revert_upgrade_incompatibleModuleId() (gas: 5464427)
[PASS] test_revert_upgrade_incompatibleUsdce() (gas: 4439708)
[PASS] test_revert_upgrade_unauthorized() (gas: 4437198)
[PASS] test_upgradeToAndCall_preservesState() (gas: 4514179)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 2.38ms (432.17µs CPU time)

Ran 3 tests for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_resolverModifier
[PASS] test_revert_reportResult_resolutionPaused() (gas: 52730)
[PASS] test_revert_reportResult_resolverPaused() (gas: 50583)
[PASS] test_revert_reportResult_wrongSender() (gas: 20846)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 1.96ms (59.58µs CPU time)

Ran 2 tests for
src/oracle/test/modules/reporters/ChainlinkReporterModule.t.sol:ChainlinkReporterModuleTest_upgrade
[PASS] test_revert_upgrade_unauthorized() (gas: 1561468)
[PASS] test_upgradeToAndCall() (gas: 1565729)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.18ms (85.58µs CPU time)

Ran 4 tests for src/oracle/test/common/OptimisticOracleModule.t.sol:OOModuleTest_forceRerequest
[PASS] test_forceRerequest() (gas: 518231)
[PASS] test_forceRerequest_resetsTrackedReward() (gas: 553499)
[PASS] test_revert_forceRerequest_notInitialized() (gas: 27102)
[PASS] test_revert_forceRerequest_unauthorized() (gas: 383290)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 8.17ms (313.63µs CPU time)

Ran 4 tests for src/oracle/test/OracleAggregator.t.sol:OracleAggregatorTest_resolve
[PASS] test_adminResolve_duringArbitration_arbitratorHookReverts_stillResolves() (gas: 794653)
[PASS] test_adminResolve_duringArbitration_clearsArbitratorState() (gas: 684243)
[PASS] test_adminResolve_outsideArbitration_skipsArbitratorHook() (gas: 514006)
[PASS] test_atomic_adminResolve_reportsAllSubconditions() (gas: 771184)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 2.81ms (835.08µs CPU time)

Ran 1 test for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_initialize
[PASS] test_revert_cannotReinitialize() (gas: 22645)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.87ms (9.71µs CPU time)

Ran 1 test for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_getPayoutCoverage
[PASS] test_revert_getPayout_invalidOutcomeIndex() (gas: 79191)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.89ms (25.63µs CPU time)

Ran 3 tests for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_viaRouter
[PASS] test_merge_viaRouter() (gas: 386794)
[PASS] test_redeem_viaRouter() (gas: 415105)
[PASS] test_split_viaRouter() (gas: 278240)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 2.12ms (235.54µs CPU time)

Ran 2 tests for src/routers/test/Router.t.sol:Router_Test_authorizeUpgrade
[PASS] test_revert_upgrade_unauthorized() (gas: 1695832)
[PASS] test_upgrade_ownerSucceeds() (gas: 27092205)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 3.14ms (1.26ms CPU time)

Ran 1 test for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_legacyPositionId
[PASS] test_getLegacyPositionId_nonMigrated() (gas: 14900)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 10.35ms (16.04µs CPU time)

Ran 2 tests for src/oracle/test/common/OptimisticOracleModule.t.sol:OOModuleTest_initializer
[PASS] test_revert_initialize_onImplementation() (gas: 2132506)
```

```
[PASS] test_revert_initialize_twice() (gas: 27073)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.24ms (66.54µs CPU time)

Ran 8 tests for
src/oracle/test/modules/reporters/ChainlinkReporterModule.t.sol:ChainlinkReporterModuleTest_view
[PASS] test_computeQuestionId() (gas: 11957)
[PASS] test_getEventWindow() (gas: 316957)
[PASS] test_getPrice() (gas: 68320)
[PASS] test_isPriceAvailable_false() (gas: 20473)
[PASS] test_isPriceAvailable_noFeedId() (gas: 12666)
[PASS] test_isPriceAvailable_true() (gas: 68090)
[PASS] test_revert_getEventWindow_notInitialized() (gas: 15584)
[PASS] test_revert_getPrice_feedIdNotSet() (gas: 15342)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 2.31ms (153.50µs CPU time)

Ran 1 test for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_migrateAmountsMismatch
[PASS] test_revert_migratePositions_operator_amountsMismatch() (gas: 28934)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.95ms (19.96µs CPU time)

Ran 4 tests for src/modules/test/BinaryMigration.t.sol:BinaryMigrationTest_softViewAliasReverts
[PASS] test_getResult_unknownCanonicalReturnsEmpty() (gas: 13328)
[PASS] test_hasResult_unknownCanonicalReturnsFalse() (gas: 12697)
[PASS] test_revert_getResult_nonCanonical() (gas: 6183)
[PASS] test_revert_hasResult_nonCanonical() (gas: 6195)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 6.08ms (1.38ms CPU time)

Ran 3 tests for src/oracle/test/OracleAggregator.t.sol:OracleAggregatorTest_voteOnce
[PASS] test_reportResult_differentModulesSameRequest() (gas: 574626)
[PASS] test_reportResult_sameModuleDifferentRequests() (gas: 553180)
[PASS] test_revert_reportResult_alreadyVoted() (gas: 498132)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 2.33ms (313.29µs CPU time)

Ran 5 tests for src/collateral/test/CollateralOfframp.t.sol:CollateralOfframpTest_admin
[PASS] test_addAdmin() (gas: 44425)
[PASS] test_removeAdmin() (gas: 34122)
[PASS] test_revert_addAdmin_unauthorized() (gas: 18344)
[PASS] test_revert_pause_unauthorized() (gas: 15109)
[PASS] test_revert_removeAdmin_unauthorized() (gas: 18396)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 585.54µs (62.67µs CPU time)

Ran 3 tests for src/routers/test/RouterSnapshots.t.sol:RouterSnapshots_Test_binary
[PASS] test_merge() (gas: 388130)
[PASS] test_redeem() (gas: 417636)
[PASS] test_splitBinary() (gas: 276723)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 2.32ms (261.79µs CPU time)

Ran 4 tests for src/routers/test/Router.t.sol:Router_Test_initialize
[PASS] test_initialize_setsOwner() (gas: 14510)
[PASS] test_revert_initialize_onImplementation() (gas: 1683748)
[PASS] test_revert_initialize_twice() (gas: 20436)
[PASS] test_revert_initialize_zeroOwner() (gas: 1770655)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 1.98ms (102.67µs CPU time)

Ran 4 tests for src/collateral/test/CollateralOfframp.t.sol:CollateralOfframpTest_pause
[PASS] test_pause_emitsEvent() (gas: 41150)
[PASS] test_revert_unwrapUSDC_paused() (gas: 205668)
[PASS] test_revert_unwrapUSDCe_paused() (gas: 205729)
[PASS] test_unpause() (gas: 261284)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 648.54µs (133.21µs CPU time)

Ran 1 test for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_merge
[PASS] test_merge() (gas: 390202)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.99ms (93.00µs CPU time)

Ran 2 tests for src/collateral/test/CollateralOfframp.t.sol:CollateralOfframpTest_unwrap
[PASS] test_unwrapUSDC() (gas: 235732)
[PASS] test_unwrapUSDCe() (gas: 235787)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 589.17µs (89.21µs CPU time)

Ran 1 test for
src/exchange/test/MatchOrdersBalanceFlows.t.sol:MatchOrdersBalanceFlowsTest_takerPositionRefund
[PASS] test_batchSellTakerExcessPositionsRefunded() (gas: 563893)
```

```
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 5.09ms (1.52ms CPU time)

Ran 5 tests for src/collateral/test/CollateralOnramp.t.sol:CollateralOnrampTest_admin
[PASS] test_addAdmin() (gas: 44402)
[PASS] test_removeAdmin() (gas: 34104)
[PASS] test_revert_addAdmin_unauthorized() (gas: 18321)
[PASS] test_revert_pause_unauthorized() (gas: 15131)
[PASS] test_revert_removeAdmin_unauthorized() (gas: 18396)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 566.92µs (60.92µs CPU time)

Ran 7 tests for src/oracle/test/common/OptimisticOracleModule.t.sol:OOModuleTest_multiOutcome
[PASS] test_createRequest_multiOutcome() (gas: 364463)
[PASS] test_priceProposed_multiOutcome_validPrice() (gas: 419487)
[PASS] test_priceSettled_multiOutcome_validPrice() (gas: 703998)
[PASS] test_revert_priceProposed_multiOutcome_invalidPrice() (gas: 419778)
[PASS] test_revert_priceProposed_multiOutcome_nonMultiple() (gas: 419683)
[PASS] test_revert_priceSettled_multiOutcome_invalidPrice() (gas: 378808)
[PASS] test_revert_priceSettled_multiOutcome_nonMultiple() (gas: 378759)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 2.83ms (668.84µs CPU time)

Ran 4 tests for src/oracle/test/integration/OracleSnapshots.t.sol:OracleSnapshotsTest_binary
[PASS] test_binary_createRequest() (gas: 242898)
[PASS] test_binary_initializeRequest() (gas: 87185)
[PASS] test_binary_priceDisputed() (gas: 247828)
[PASS] test_binary_priceSettled() (gas: 368513)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 2.47ms (323.67µs CPU time)

Ran 4 tests for src/collateral/test/CollateralOnramp.t.sol:CollateralOnrampTest_pause
[PASS] test_pause_emitsEvent() (gas: 41172)
[PASS] test_revert_wrapUSDC_paused() (gas: 117294)
[PASS] test_revert_wrapUSDCe_paused() (gas: 117261)
[PASS] test_unpause() (gas: 189712)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 597.67µs (91.62µs CPU time)

Ran 2 tests for src/routers/test/RouterBinaryModule.t.sol:RouterBinaryModuleTest_merge
[PASS] test_merge() (gas: 412044)
[PASS] test_merge_anyCondition() (gas: 201740)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.04ms (158.42µs CPU time)

Ran 4 tests for src/routers/test/RouterSnapshots.t.sol:RouterSnapshots_Test_bridge
[PASS] test_bridgeCollateral() (gas: 406787)
[PASS] test_bridgeCollateralTo() (gas: 408535)
[PASS] test_bridgePositions() (gas: 687810)
[PASS] test_bridgePositionsTo() (gas: 689548)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 2.53ms (423.50µs CPU time)

Ran 2 tests for src/collateral/test/CollateralOnramp.t.sol:CollateralOnrampTest_wrap
[PASS] test_wrapUSDC() (gas: 161372)
[PASS] test_wrapUSDCe() (gas: 161408)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 552.54µs (58.50µs CPU time)

Ran 3 tests for src/routers/test/RouterBinaryModule.t.sol:RouterBinaryModuleTest_events
[PASS] test_merge_emitsPositionsMerged() (gas: 410396)
[PASS] test_redeem_emitsPositionRedeemed() (gas: 438060)
[PASS] test_split_emitsPositionSplit() (gas: 276169)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 3.64ms (1.71ms CPU time)

Ran 2 tests for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_prepareEventCoverage
[PASS] test_constructor_zeroNegRiskAdapter() (gas: 5444400)
[PASS] test_getEventId_isDeterministic() (gas: 14252)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 5.43ms (129.29µs CPU time)

Ran 4 tests for src/oracle/test/common/OptimisticOracleModule.t.sol:OOModuleTest_priceDisputed
[PASS] test_priceDisputed_ignoresStaleRequest() (gas: 543558)
[PASS] test_priceDisputed_reRequests() (gas: 546971)
[PASS] test_priceDisputed_reRequests_preservesRewardAfterCanonicalDispute() (gas: 601991)
[PASS] test_revert_onlyOptimisticOracle() (gas: 20611)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 3.16ms (363.79µs CPU time)

Ran 5 tests for src/oracle/test/integration/OracleSnapshots.t.sol:OracleSnapshotsTest_negRisk
[PASS] test_negRisk_createRequest() (gas: 243196)
[PASS] test_negRisk_initializeRequest() (gas: 87387)
```

```
[PASS] test_negRisk_priceDisputed() (gas: 248095)
[PASS] test_negRisk_priceSettled() (gas: 467693)
[PASS] test_negRisk_resolveResult() (gas: 469037)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 2.95ms (450.75µs CPU time)

Ran 7 tests for src/exchange/test/MatchOrders.t.sol:MatchOrdersTest_complementaryTakerSell
[PASS] test_complementaryTakerSell_withFees() (gas: 610835)
[PASS] test_complementaryTakerSell_zeroFee() (gas: 553584)
[PASS] test_complementaryTakerSell_zeroMakerFee() (gas: 595991)
[PASS] test_revert_complementaryTakerSell_feePathFillExceedsTakerFill() (gas: 528066)
[PASS] test_revert_complementaryTakerSell_makerFeeOnlyNotCrossing() (gas: 458109)
[PASS] test_revert_complementaryTakerSell_zeroFeeFillExceedsTakerFill() (gas: 526390)
[PASS] test_revert_complementaryTakerSell_zeroFeeNotCrossing() (gas: 458152)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 11.27ms (9.03ms CPU time)

Ran 6 tests for src/collateral/test/PermissionedRamp.t.sol:PermissionedRampTest_admin
[PASS] test_addAdmin() (gas: 44403)
[PASS] test_addWitness() (gas: 207073)
[PASS] test_removeAdmin() (gas: 34122)
[PASS] test_removeWitness() (gas: 133768)
[PASS] test_revert_addAdmin_unauthorized() (gas: 18344)
[PASS] test_revert_removeAdmin_unauthorized() (gas: 18374)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 886.04µs (323.79µs CPU time)
```

```
Ran 19 tests for src/utils/test/AutoRedeemer.t.sol:AutoRedeemerTest_redeem
[PASS] test_redeemAllUnapproved() (gas: 545106)
[PASS] test_redeemBatch() (gas: 787806)
[PASS] test_redeemBatchManyUsers() (gas: 3575927)
[PASS] test_redeemBinary() (gas: 450705)
[PASS] test_redeemCombinatorial() (gas: 568786)
[PASS] test_redeemDoesNotForwardStrandedBalance() (gas: 477241)
[PASS] test_redeemLegacyBinary() (gas: 490346)
[PASS] test_redeemLegacyBinaryDoesNotForwardStrandedBalance() (gas: 471930)
[PASS] test_redeemLegacyBinarySkipsUnapproved() (gas: 803874)
[PASS] test_redeemLegacyNegRisk() (gas: 851286)
[PASS] test_redeemLegacyNegRiskSkipsUnapproved() (gas: 1155646)
[PASS] test_redeemNegRisk() (gas: 531806)
[PASS] test_redeemSkipsUnapproved() (gas: 699676)
[PASS] test_redeemSkipsUnapprovedFirst() (gas: 647783)
[PASS] test_redeemSkipsUnapprovedInMiddle() (gas: 1039389)
[PASS] test_revertLegacyBinaryLengthMismatch() (gas: 19078)
[PASS] test_revertLegacyNegRiskLengthMismatch() (gas: 19068)
[PASS] test_revertLengthMismatch() (gas: 19032)
[PASS] test_revertUnauthorized() (gas: 18599)
Suite result: ok. 19 passed; 0 failed; 0 skipped; finished in 10.41ms (8.11ms CPU time)

Ran 5 tests for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_reportResult
[PASS] test_allNoResolvesSyntheticOther() (gas: 360352)
[PASS] test_finalYesOutcome() (gas: 355588)
[PASS] test_reportResult() (gas: 179975)
[PASS] test_revert_multipleYesOutcomes() (gas: 232577)
[PASS] test_revert_unauthorized() (gas: 23717)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 2.10ms (228.42µs CPU time)

Ran 2 tests for src/routers/test/BridgeRouter.t.sol:BridgeRouterTest_bridgeCollateral
[PASS] test_bridgeCollateralTo_emits_explicit_recipient() (gas: 410602)
[PASS] test_emits_initiator_event() (gas: 408899)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.75ms (697.96µs CPU time)

Ran 2 tests for src/collateral/test/PermissionedRamp.t.sol:PermissionedRampTest_pause
[PASS] test_revert_unwrap_paused() (gas: 237517)
[PASS] test_revert_wrap_paused() (gas: 124862)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 798.50µs (256.79µs CPU time)

Ran 6 tests for src/oracle/test/common/OptimisticOracleModule.t.sol:OOModuleTest_priceProposed
[PASS] test_priceProposed_validNo() (gas: 421480)
[PASS] test_priceProposed_validYes() (gas: 441371)
[PASS] test_revert_priceProposed_invalidPrice() (gas: 441660)
[PASS] test_revert_priceProposed_negativePrice() (gas: 441683)
[PASS] test_revert_priceProposed_onlyOptimisticOracle() (gas: 20574)
[PASS] test_revert_priceProposed_p4() (gas: 441680)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 2.64ms (479.63µs CPU time)
```

```
Ran 1 test for src/collateral/dev/CollateralSetup.t.sol:CollateralSetupTest
[PASS] test_setup() (gas: 36933)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 4.79ms (112.08µs CPU time)

Ran 3 tests for src/routers/test/RouterSnapshots.t.sol:RouterSnapshots_Test_negRisk
[PASS] test_convert() (gas: 6955316)
[PASS] test_horizontalMerge_256() (gas: 10351197)
[PASS] test_horizontalSplit_256() (gas: 6222631)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 4.95ms (2.81ms CPU time)

Ran 2 tests for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_view
[PASS] test_getConditionId(bytes) (runs: 256, µ: 12087, ~: 12047)
[PASS] test_getPositionId(bytes32,uint256) (runs: 256, µ: 415, ~: 415)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 9.92ms (12.82ms CPU time)

Ran 7 tests for src/collateral/test/PermissionedRamp.t.sol:PermissionedRampTest_signature
[PASS] test_revert_unwrap_expiredDeadline() (gas: 211550)
[PASS] test_revert_unwrap_invalidWitness() (gas: 216452)
[PASS] test_revert_wrap_expiredDeadline() (gas: 98025)
[PASS] test_revert_wrap_invalidNonce() (gas: 120271)
[PASS] test_revert_wrap_invalidWitness() (gas: 124124)
[PASS] test_revert_wrap_replaySignature() (gas: 218032)
[PASS] test_revert_wrap_signerNotWitness() (gas: 122073)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 1.40ms (841.54µs CPU time)

Ran 1 test for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_reportResultCoverage
[PASS] test_revert_migratePositions_operator_arrayMismatch() (gas: 28957)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.87ms (19.88µs CPU time)

Ran 2 tests for src/utils/test/AutoRedeemer.t.sol:AutoRedeemerTest_upgrade
[PASS] test_revert_upgradeUnauthorized() (gas: 2934091)
[PASS] test_upgradeToAndCall_preservesState() (gas: 2959035)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.39ms (166.04µs CPU time)

Ran 2 tests for src/collateral/test/PermissionedRamp.t.sol:PermissionedRampTest_unwrap
[PASS] test_unwrapUSDC() (gas: 264966)
[PASS] test_unwrapUSDCe() (gas: 265021)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 957.46µs (419.21µs CPU time)

Ran 2 tests for src/routers/test/BridgeRouter.t.sol:BridgeRouterTest_bridgePositions
[PASS] test_bridgePositionsTo_emits_explicit_recipient() (gas: 693552)
[PASS] test_emits_initiator_event() (gas: 691854)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.86ms (832.79µs CPU time)

Ran 1 test for src/modules/test/BinaryMigration.t.sol:BinaryMigrationTest_aliasKeyRegression
[PASS] test_revert_resolveMigrationCondition_aliasOutcomeByte() (gas: 86991)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.33ms (25.42µs CPU time)

Ran 3 tests for src/collateral/test/PermissionedRamp.t.sol:PermissionedRampTest_wrap
[PASS] test_incrementsNonce() (gas: 223762)
[PASS] test_wrapUSDC() (gas: 190111)
[PASS] test_wrapUSDCe() (gas: 190147)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 962.75µs (425.37µs CPU time)

Ran 8 tests for
src/oracle/test/modules/reporters/ChainlinkReporterModule.t.sol:ChainlinkReporterModuleTest_admin
[PASS] test_pause() (gas: 43370)
[PASS] test_revert_pause_notAdmin() (gas: 17682)
[PASS] test_revert_setFeedId_notAdmin() (gas: 17801)
[PASS] test_revert_setPriceSource_notAdmin() (gas: 19130)
[PASS] test_revert_unpause_notAdmin() (gas: 46492)
[PASS] test_setFeedId() (gas: 44361)
[PASS] test_setPriceSource() (gas: 30758)
[PASS] test_unpause() (gas: 33731)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 2.28ms (158.42µs CPU time)

Ran 5 tests for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_resolveConditionToNo
[PASS] test_resolveConditionToNo_afterOneYes() (gas: 363115)
[PASS] test_resolveConditionToNo_emitsModuleAsResolver() (gas: 245191)
[PASS] test_revert_invalidConditionIndex() (gas: 25138)
[PASS] test_revert_invalidEventId() (gas: 19988)
```

```
[PASS] test_revert_reportResult_beforeAllYes() (gas: 114645)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 2.63ms (745.17µs CPU time)

Ran 10 tests for src/oracle/test/common/OptimisticOracleModule.t.sol:OOModuleTest_priceSettled
[PASS] test_priceSettled_afterAdminOverride_silentlyReturns() (gas: 535286)
[PASS] test_priceSettled_noPrice_resolvesAggregator() (gas: 530011)
[PASS] test_priceSettled_p3_resolves5050() (gas: 553201)
[PASS] test_priceSettled_p4AfterAdminOverride_noReRequest() (gas: 530137)
[PASS] test_priceSettled_p4_reRequests() (gas: 555043)
[PASS] test_priceSettled_p4_reRequests_preservesRewardAfterDispute() (gas: 769475)
[PASS] test_priceSettled_staleP4_skipsReRequest() (gas: 464233)
[PASS] test_priceSettled_validPrice_resolvesAggregator() (gas: 530066)
[PASS] test_revert_priceSettled_invalidPrice() (gas: 422839)
[PASS] test_revert_priceSettled_negativePrice() (gas: 422862)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 3.72ms (1.54ms CPU time)

Ran 2 tests for src/collateral/test/CollateralToken.t.sol:CollateralTokenTest_burn
[PASS] test_burn() (gas: 54211)
[PASS] test_revert_unauthorized() (gas: 19821)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 3.58ms (73.04µs CPU time)

Ran 1 test for src/modules/test/BinaryMigration.t.sol:BinaryMigrationTest_cantina10
[PASS] test_migratePositions_doesNotInvokeReceiverHook() (gas: 403940)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.61ms (302.63µs CPU time)

Ran 2 tests for src/exchange/test/ExchangeAdmin.t.sol:ExchangeAdminTest_setup
[PASS] test_initialState() (gas: 38991)
[PASS] test_initializerDefaults() (gas: 4688557)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.37ms (143.08µs CPU time)

Ran 3 tests for src/test/integration/StorageSlots.t.sol:StorageSlotsTest_binaryModule
[PASS] test_implementationStoredInSlot() (gas: 7379)
[PASS] test_ownerStoredInSlot() (gas: 7412)
[PASS] test_roleStoredInSlot() (gas: 7453)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 4.64ms (64.25µs CPU time)

Ran 3 tests for
src/oracle/test/modules/reporters/ChainlinkReporterModule.t.sol:ChainlinkReporterModuleTest_initializeReq
uest
[PASS] test_initializeRequest() (gas: 316284)
[PASS] test_revert_initializeRequest_notBinaryEvent() (gas: 52752)
[PASS] test_revert_initializeRequest_windowAlreadyEnded() (gas: 53347)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 2.17ms (99.83µs CPU time)

Ran 2 tests for src/collateral/test/CollateralToken.t.sol:CollateralTokenTest_initialize
[PASS] test_initialize() (gas: 14593)
[PASS] test_revert_alreadyInitialized() (gas: 20419)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 1.66ms (42.08µs CPU time)

Ran 9 tests for src/exchange/test/MatchOrders.t.sol:MatchOrdersTest_feeValidation
[PASS] test_maxFeeRateZero_disablesEnforcement() (gas: 5219123)
[PASS] test_revert_complementaryTakerFeeValidatedAgainstActualFill() (gas: 5184066)
[PASS] test_revert_feeExceedsMaxRate_buyOrder() (gas: 5184605)
[PASS] test_revert_feeExceedsMaxRate_sellOrder() (gas: 5210194)
[PASS] test_revert_feeExceedsProceeds_batchSell() (gas: 5306518)
[PASS] test_revert_feeExceedsProceeds_complementarySell() (gas: 5186216)
[PASS] test_revert_makerFeeExceedsMaxRate() (gas: 5109125)
[PASS] test_takerSellSurplusFeeValidationUsesActualProceeds() (gas: 597876)
[PASS] test_validateFee_externalView() (gas: 10538)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 7.19ms (3.36ms CPU time)

Ran 1 test for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_resolveConditionToNoPauses
[PASS] test_revert_resolveConditionToNo_resolutionPaused() (gas: 214940)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.99ms (40.88µs CPU time)

Ran 3 tests for src/test/integration/StorageSlots.t.sol:StorageSlotsTest_exchange
[PASS] test_implementationStoredInSlot() (gas: 7379)
[PASS] test_ownerStoredInSlot() (gas: 7412)
[PASS] test_roleStoredInSlot() (gas: 7453)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 1.54ms (21.33µs CPU time)

Ran 2 tests for src/exchange/test/ExchangeAdmin.t.sol:ExchangeAdminTest_tradingPausedEvents
```

```
[PASS] test_pauseTrading_emitsEvent() (gas: 42363)
[PASS] test_unpauseTrading_emitsEvent() (gas: 32124)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.22ms (27.25µs CPU time)

Ran 2 tests for src/collateral/test/CollateralToken.t.sol:CollateralTokenTest_mint
[PASS] test_mint() (gas: 69468)
[PASS] test_revert_unauthorized() (gas: 20029)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 1.66ms (52.63µs CPU time)

Ran 3 tests for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_resolverRole
[PASS] test_addResolver_thenReport() (gas: 206872)
[PASS] test_reportResult_idempotent() (gas: 186651)
[PASS] test_revert_unauthorized() (gas: 23702)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 1.98ms (125.08µs CPU time)

Ran 3 tests for src/test/integration/StorageSlots.t.sol:StorageSlotsTest_negRiskModule
[PASS] test_implementationStoredInSlot() (gas: 7379)
[PASS] test_ownerStoredInSlot() (gas: 7412)
[PASS] test_roleStoredInSlot() (gas: 7453)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 1.52ms (20.38µs CPU time)

Ran 3 tests for src/test/integration/StorageSlots.t.sol:StorageSlotsTest_bridgeRouter
[PASS] test_implementationStoredInSlot() (gas: 7379)
[PASS] test_ownerStoredInSlot() (gas: 7412)
[PASS] test_roleStoredInSlot() (gas: 44075)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 3.32ms (57.46µs CPU time)

Ran 10 tests for src/modules/test/BinaryMigration.t.sol:BinaryMigrationTest_migrate
[PASS] test_migrate() (gas: 489383)
[PASS] test_migrate_asymmetricComplementaryBalances_sameCall() (gas: 482111)
[PASS] test_migrate_fromOperator() (gas: 498886)
[PASS] test_migrate_repeatedConditionEntries_clampsToCurrentBalance() (gas: 541491)
[PASS] test_prepareMigrationCondition_emitsMigrationConditionRegistered() (gas: 87425)
[PASS] test_revert_InvalidFromAddress() (gas: 355628)
[PASS] test_revert_alreadyResolved() (gas: 380470)
[PASS] test_revert_amountsLengthMismatch() (gas: 91447)
[PASS] test_revert_invalidArrayLength() (gas: 91448)
[PASS] test_revert_migrationNotRegistered() (gas: 198661)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 4.28ms (2.91ms CPU time)

Ran 2 tests for src/oracle/test/common/OptimisticOracleModule.t.sol:OOModuleTest_setOracleAllowance
[PASS] test_revert_setOracleAllowance_unauthorized() (gas: 19149)
[PASS] test_setOracleAllowance() (gas: 52313)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 5.03ms (156.12µs CPU time)

Ran 2 tests for src/exchange/test/ExchangeAdmin.t.sol:ExchangeAdminTest_upgrade
[PASS] test_revert_upgrade_unauthorized() (gas: 4535820)
[PASS] test_upgradeToAndCall() (gas: 4540148)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.37ms (205.21µs CPU time)

Ran 1 test for src/collateral/test/CollateralToken.t.sol:CollateralTokenTest_permit2
[PASS] test_permit2NoInfiniteAllowance() (gas: 14823)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.60ms (19.17µs CPU time)

Ran 4 tests for src/exchange/test/MatchOrders.t.sol:MatchOrdersTest_merge
[PASS] test_merge() (gas: 734254)
[PASS] test_mergeFees() (gas: 760284)
[PASS] test_revert_mergeCannotConsumeMoreThanTakerFill() (gas: 598903)
[PASS] test_revert_mixedBatchSellCannotConsumeMoreThanTakerFill() (gas: 759303)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 3.86ms (1.59ms CPU time)

Ran 4 tests for src/test/integration/StorageSlots.t.sol:StorageSlotsTest_oracleAggregator
[PASS] test_globalPausedStoredInSlot() (gas: 66127)
[PASS] test_implementationStoredInSlot() (gas: 7401)
[PASS] test_ownerStoredInSlot() (gas: 7434)
[PASS] test_roleStoredInSlot() (gas: 44039)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 1.54ms (33.25µs CPU time)

Ran 4 tests for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_upgrade
[PASS] test_revert_upgrade_incompatibleModuleId() (gas: 4437960)
[PASS] test_revert_upgrade_incompatibleUsdce() (gas: 5470505)
[PASS] test_revert_upgrade_unauthorized() (gas: 5467984)
```

```
[PASS] test_upgradeToAndCall_preservesState() (gas: 5612347)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 2.34ms (475.92µs CPU time)

Ran 2 tests for src/oracle/test/common/OptimisticOracleModule.t.sol:OOModuleTest_withdrawERC20
[PASS] test_revert_withdrawERC20_unauthorized() (gas: 19989)
[PASS] test_withdrawERC20() (gas: 467843)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.22ms (101.25µs CPU time)

Ran 7 tests for
src/oracle/test/libraries/OptimisticOraclePayoutLib.t.sol:OptimisticOraclePayoutLibTest_priceToPayouts
[PASS] test_noPrice_singleOutcome() (gas: 6595)
[PASS] test_numerical_winnerIndex0() (gas: 7260)
[PASS] test_numerical_winnerIndex2() (gas: 7235)
[PASS] test_p3Price_singleOutcome() (gas: 6736)
[PASS] test_revert_zeroOutcomes() (gas: 10347)
[PASS] test_revert_zeroOutcomes_nonZeroPrice() (gas: 10354)
[PASS] test_yesPrice_singleOutcome() (gas: 6654)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 126.75µs (59.21µs CPU time)

Ran 3 tests for src/exchange/test/ExchangeAdmin.t.sol:ExchangeAdminTest_userPauseBlockInterval
[PASS] test_revert_exceedsMaxPauseInterval() (gas: 17791)
[PASS] test_setUserPauseBlockInterval() (gas: 26835)
[PASS] test_setUserPauseBlockInterval_maxAllowed() (gas: 24983)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 2.21ms (35.04µs CPU time)

Ran 6 tests for src/test/integration/StorageSlots.t.sol:StorageSlotsTest_positionManager
[PASS] test_erc1155ApprovalStoredInSlot() (gas: 42328)
[PASS] test_erc1155BalanceStoredInSlot() (gas: 50079)
[PASS] test_handoverStoredInSlot() (gas: 39187)
[PASS] test_implementationStoredInSlot() (gas: 7401)
[PASS] test_ownerStoredInSlot() (gas: 7434)
[PASS] test_roleStoredInSlot() (gas: 7476)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 1.57ms (58.00µs CPU time)

Ran 3 tests for src/positionManager/test/PositionManager.t.sol:PositionManagerTest_addModule
[PASS] test_revert_addModule_alreadyRegistered() (gas: 25220)
[PASS] test_revert_addModule_invalidModuleId() (gas: 278687)
[PASS] test_revert_removeModule_notRegistered() (gas: 19936)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 9.57ms (98.58µs CPU time)

Ran 2 tests for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_viaRouter
[PASS] test_horizontalMerge_viaRouter() (gas: 468984)
[PASS] test_horizontalSplit_viaRouter() (gas: 319120)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.07ms (183.67µs CPU time)

Ran 7 tests for src/test/integration/StorageSlots.t.sol:StorageSlotsTest_collateralToken
[PASS] test_allowanceStoredInSlot() (gas: 42331)
[PASS] test_balanceStoredInSlot() (gas: 91557)
[PASS] test_implementationStoredInSlot() (gas: 7379)
[PASS] test_noncesStoredInSlot() (gas: 13351)
[PASS] test_ownerStoredInSlot() (gas: 7412)
[PASS] test_roleStoredInSlot() (gas: 44061)
[PASS] test_totalSupplyStoredInSlot() (gas: 91341)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 3.69ms (170.46µs CPU time)

Ran 8 tests for src/collateral/test/CollateralToken.t.sol:CollateralTokenTest_roles
[PASS] test_addMinter() (gas: 44836)
[PASS] test_addWrapper() (gas: 44882)
[PASS] test_removeMinter() (gas: 35863)
[PASS] test_removeWrapper() (gas: 35847)
[PASS] test_revert_addMinter_unauthorized() (gas: 19848)
[PASS] test_revert_addWrapper_unauthorized() (gas: 19849)
[PASS] test_revert_removeMinter_unauthorized() (gas: 19816)
[PASS] test_revert_removeWrapper_unauthorized() (gas: 19848)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 3.07ms (92.38µs CPU time)

Ran 2 tests for src/oracle/test/common/OptimisticOracleModule.t.sol:OOModuleTest_upgrade
[PASS] test_revert_upgrade_unauthorized() (gas: 2137698)
[PASS] test_upgradeToAndCall() (gas: 2141960)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 3.87ms (103.71µs CPU time)

Ran 2 tests for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_redeem
```

```
[PASS] test_redeem(bool) (runs: 256, μ: 416736, ~: 416736)
[PASS] test_redeem_emitsPositionRedeemed() (gas: 395524)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 32.37ms (29.85ms CPU time)

Ran 3 tests for src/test/integration/StorageSlots.t.sol:StorageSlotsTest_router
[PASS] test_implementationStoredInSlot() (gas: 7379)
[PASS] test_ownerStoredInSlot() (gas: 7412)
[PASS] test_roleStoredInSlot() (gas: 44075)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 1.54ms (22.21µs CPU time)

Ran 4 tests for src/exchange/test/MatchOrders.t.sol:MatchOrdersTest_mint
[PASS] test_mint() (gas: 536763)
[PASS] test_mintConsumesLimitFill_whenFundedAtExecutionPrice() (gas: 539671)
[PASS] test_mintFees() (gas: 562628)
[PASS] test_mintRefundedTakerEventReportsNetFill() (gas: 570243)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 3.94ms (1.75ms CPU time)

Ran 15 tests for src/oracle/test/OracleAggregator.t.sol:OracleAggregatorTest_additionalCoverage
[PASS] test_getRequestState_activeForValidCondition() (gas: 392465)
[PASS] test_getRequestState_noneForNonExistent() (gas: 17451)
[PASS] test_resolveResult_alreadyResolved_noop() (gas: 511030)
[PASS] test_revert_atomic_requestRejectsChildConditionId() (gas: 417772)
[PASS] test_revert_getRequestState_nonCanonicalRequestId() (gas: 13637)
[PASS] test_revert_initializeRequest_alreadyExists() (gas: 390237)
[PASS] test_revert_initializeRequest_atomicTypeRejectsWrongResultLength() (gas: 58771)
[PASS] test_revert_initializeRequest_nonCanonicalEventId() (gas: 38304)
[PASS] test_revert_initializeRequest_notOperator() (gas: 66704)
[PASS] test_revert_pauseOracle_notAdmin() (gas: 17711)
[PASS] test_revert_reportResult_invalidConditionIndex() (gas: 417837)
[PASS] test_revert_reportResult_requestNotFound() (gas: 20680)
[PASS] test_revert_resolveResult_notAuthorized() (gas: 395462)
[PASS] test_revert_unpauseOracle_notAdmin() (gas: 46573)
[PASS] test_revert_upgradeToAndCall_notOwner() (gas: 2681467)
Suite result: ok. 15 passed; 0 failed; 0 skipped; finished in 2.63ms (646.21µs CPU time)

Ran 2 tests for src/exchange/test/ExchangeAdmin.t.sol:ExchangeAdminTest_userPauseEvents
[PASS] test_pauseUser_emitsEvent() (gas: 44378)
[PASS] test_unpauseUser_emitsEvent() (gas: 32219)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.27ms (28.58µs CPU time)

Ran 5 tests for src/positionManager/test/PositionManager.t.sol:PositionManagerTest_crossModuleBurn
[PASS] test_crossModuleBatchBurn() (gas: 94924)
[PASS] test_crossModuleBurn() (gas: 71291)
[PASS] test_crossModuleMint_revoked() (gas: 48065)
[PASS] test_revert_crossModuleBatchBurn_unauthorized() (gas: 83063)
[PASS] test_revert_crossModuleBurn_unauthorized() (gas: 53369)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 2.12ms (326.96µs CPU time)

Ran 3 tests for src/test/integration/StorageSlots.t.sol:StorageSlotsTest_combinatorialModule
[PASS] test_implementationStoredInSlot() (gas: 7379)
[PASS] test_ownerStoredInSlot() (gas: 7412)
[PASS] test_roleStoredInSlot() (gas: 7453)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 2.14ms (20.96µs CPU time)

Ran 5 tests for src/test/integration/StorageSlots.t.sol:StorageSlotsTest_soladySharedSlots
[PASS] test_erc1967ImplementationSlot() (gas: 329)
[PASS] test_handoverSlotSeed() (gas: 2275581)
[PASS] test_initializableSlot() (gas: 272)
[PASS] test_ownerSlot() (gas: 295)
[PASS] test_roleSlotSeed() (gas: 251)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 1.58ms (80.12µs CPU time)

Ran 2 tests for src/collateral/test/CollateralToken.t.sol:CollateralTokenTest_upgrade
[PASS] test_revert_unauthorized() (gas: 1350970)
[PASS] test_upgradeToAndCall() (gas: 1355218)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 637.29µs (76.25µs CPU time)

Ran 4 tests for src/modules/test/BinaryModule.t.sol:BinaryModuleTest_reportResult
[PASS] test_reportResult() (gas: 77655)
[PASS] test_reportResult_idempotent() (gas: 83604)
[PASS] test_revert_invalidSum() (gas: 27778)
[PASS] test_revert_unauthorized() (gas: 20846)
```

```
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 2.10ms (90.63µs CPU time)

Ran 5 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_convertToYesBasket
[PASS] test_convertToYesBasket() (gas: 554578)
[PASS] test_convertToYesBasket_singleCondition() (gas: 291719)
[PASS] test_revert_convertToYesBasket_requiresNoSide() (gas: 263079)
[PASS] test_revert_convertToYesBasket_unprepared() (gas: 190107)
[PASS] test_revert_convertToYesBasket_wrongToLength() (gas: 263562)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 2.56ms (400.63µs CPU time)

Ran 3 tests for src/collateral/test/CollateralToken.t.sol:CollateralTokenTest_validAsset
[PASS] test_revert_invalidAsset() (gas: 20165)
[PASS] test_validAsset_usdc() (gas: 126369)
[PASS] test_validAsset_usdce() (gas: 126338)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 597.88µs (45.33µs CPU time)

Ran 9 tests for src/positionManager/test/UnsafeTransfer.t.sol:UnsafeTransferTest_unsafeBatchTransferFrom
[PASS] test_approved() (gas: 113695)
[PASS] test_basic() (gas: 88919)
[PASS] test_emitsTransferBatch() (gas: 88202)
[PASS] test_emptyArrays() (gas: 21766)
[PASS] test_revert_insufficientBalance() (gas: 23737)
[PASS] test_revert_lengthMismatch() (gas: 23618)
[PASS] test_revert_notApproved() (gas: 23659)
[PASS] test_revert_toZeroAddress() (gas: 19370)
[PASS] test_toNonReceiver() (gas: 96161)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 791.88µs (148.58µs CPU time)

Ran 6 tests for src/oracle/test/OracleAggregator.t.sol:OracleAggregatorTest_configValidation
[PASS] test_initializeRequest_noDisputeConfig() (gas: 200944)
[PASS] test_revert_initializeRequest_zeroArbitratorWithDisputers() (gas: 70235)
[PASS] test_revert_initializeRequest_zeroDisputerThresholdWithDisputers() (gas: 72167)
[PASS] test_revert_initializeRequest_zeroLivenessWithDisputers() (gas: 72456)
[PASS] test_revert_initializeRequest_zeroReporterThreshold() (gas: 64392)
[PASS] test_revert_initializeRequest_zeroTargetContract() (gas: 64055)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 2.10ms (183.04µs CPU time)

Ran 4 tests for src/collateral/test/CollateralToken.t.sol:CollateralTokenTest_view
[PASS] test_decimals() (gas: 10311)
[PASS] test_immutables() (gas: 18942)
[PASS] test_name() (gas: 14001)
[PASS] test_symbol() (gas: 14011)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 591.13µs (37.00µs CPU time)

Ran 1 test for src/exchange/test/MatchOrders.t.sol:MatchOrdersTest_partialFill
[PASS] test_partialFill() (gas: 538800)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.59ms (334.13µs CPU time)

Ran 3 tests for src/exchange/test/ExchangeAdmin.t.sol:ExchangeAdminTest_validateOrder
[PASS] test_revert_validateOrder_invalidSignature() (gas: 29191)
[PASS] test_revert_validateOrder_userPaused() (gas: 57408)
[PASS] test_validateOrder_validOrder() (gas: 32780)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 2.52ms (243.46µs CPU time)

Ran 3 tests for src/test/integration/StorageSlots.t.sol:StorageSlotsTest_ctfRouter
[PASS] test_implementationStoredInSlot() (gas: 7379)
[PASS] test_ownerStoredInSlot() (gas: 7412)
[PASS] test_roleStoredInSlot() (gas: 44052)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 1.53ms (24.04µs CPU time)

Ran 4 tests for src/positionManager/test/PositionManager.t.sol:PositionManagerTest_crossModuleMint
[PASS] test_crossModuleBatchMint() (gas: 116917)
[PASS] test_crossModuleMint() (gas: 87606)
[PASS] test_revert_crossModuleBatchMint_unauthorized() (gas: 31234)
[PASS] test_revert_crossModuleMint_unauthorized() (gas: 28644)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 1.88ms (197.38µs CPU time)

Ran 2 tests for src/oracle/test/OracleAggregator.t.sol:OracleAggregatorTest_coverage
[PASS] test_initializeRequest_successPath() (gas: 388180)
[PASS] test_revert_finalizeConditions_targetReverts() (gas: 675033)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.09ms (187.38µs CPU time)
```

```
Ran 1 test for src/positionManager/test/PositionManager.t.sol:PositionManagerTest_deployment
[PASS] test_deployment() (gas: 23514)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.76ms (30.92µs CPU time)

Ran 4 tests for src/exchange/test/MatchOrders.t.sol:MatchOrdersTest_proxyAndSafeSignatures
[PASS] test_polyGnosisSafeSignature() (gas: 618948)
[PASS] test_polyProxySignature() (gas: 588978)
[PASS] test_revert_polyGnosisSafeSignature_unrelatedMaker() (gas: 121623)
[PASS] test_revert_polyProxySignature_unrelatedMaker() (gas: 84110)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 3.15ms (931.88µs CPU time)

Ran 1 test for src/oracle/test/OracleAggregator.t.sol:OracleAggregatorTest_dispute
[PASS] test_dispute_triggersArbitration() (gas: 627696)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.04ms (116.63µs CPU time)

Ran 1 test for src/positionManager/test/PositionManager.t.sol:PositionManagerTest_getPayout
[PASS] test_revert_getPayout_moduleNotRegistered() (gas: 15654)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.70ms (28.38µs CPU time)

Ran 1 test for src/libraries/test/Ids.t.sol:EventIdLibTest_encodeFromData
[PASS] test_negRiskEventId(uint256,bytes) (runs: 256, µ: 1514, ~: 1508)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 3.70ms (3.42ms CPU time)

Ran 1 test for src/oracle/test/common/OptimisticOracleModule.t.sol:OOModuleTest_constructor
[PASS] test_constructor() (gas: 25291)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.25ms (16.58µs CPU time)

Ran 4 tests for src/libraries/test/Ids.t.sol:IdsTest_canonicalBoundaries
[PASS] test_conditionIdFrom_canonicalMax() (gas: 370)
[PASS] test_conditionIdFrom_zero() (gas: 341)
[PASS] test_eventIdFrom_canonicalMax() (gas: 567)
[PASS] test_eventIdFrom_zero() (gas: 502)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 303.83µs (19.46µs CPU time)

Ran 2 tests for src/modules/test/NegRiskMigration.t.sol:NegRiskMigrationTest_aliasKeyRegression
[PASS] test_revert_resolveConditionToNo_aliasOutcomeByte() (gas: 943864)
[PASS] test_revert_resolveMigrationCondition_aliasOutcomeByte() (gas: 944110)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 4.97ms (3.10ms CPU time)

Ran 5 tests for src/collateral/test/CollateralToken.t.sol:CollateralTokenTest_wrap
[PASS] test_revert_invalidAsset(address) (runs: 256, µ: 27053, ~: 27053)
[PASS] test_revert_unauthorized() (gas: 24257)
[PASS] test_revert_wrap_noPreTransfer() (gas: 75880)
[PASS] test_wrapUSDC() (gas: 135075)
[PASS] test_wrapUSDCe() (gas: 135131)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 5.50ms (4.94ms CPU time)

Ran 4 tests for src/positionManager/test/PositionManager.t.sol:PositionManagerTest_initRoles
[PASS] test_creatorAction_success() (gas: 35635)
[PASS] test_operatorAction_success() (gas: 35636)
[PASS] test_revert_creatorAction_unauthorized() (gas: 12875)
[PASS] test_revert_operatorAction_unauthorized() (gas: 12876)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 1.15ms (79.21µs CPU time)

Ran 1 test for src/oracle/test/OracleAggregator.t.sol:OracleAggregatorTest_finalize
[PASS] test_incremental_finalize_reportsTranslatedPayout() (gas: 738750)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.05ms (134.42µs CPU time)

Ran 7 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_extract
[PASS] test_extract() (gas: 477078)
[PASS] test_extract_fullDecomposition() (gas: 657570)
[PASS] test_revert_extract_requiresNoSide() (gas: 262896)
[PASS] test_revert_extract_unprepared() (gas: 189918)
[PASS] test_revert_extract_wrongToLength() (gas: 262960)
[PASS] test_revert_indexOutOfRange() (gas: 242277)
[PASS] test_revert_singleCondition() (gas: 221461)
Suite result: ok. 7 passed; 0 failed; 0 skipped; finished in 6.68ms (4.61ms CPU time)

Ran 1 test for src/positionManager/test/PositionManager.t.sol:PositionManagerTest_merge
[PASS] test_merge() (gas: 112638)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 709.83µs (45.79µs CPU time)
```

```
Ran 3 tests for src/oracle/test/common/OptimisticOracleModule.t.sol:OOModuleTest_createRequest
[PASS] test_createRequest() (gas: 487277)
[PASS] test_revert_createRequest_alreadyInitialized() (gas: 377751)
[PASS] test_revert_createRequest_unauthorized() (gas: 172825)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 2.40ms (228.04µs CPU time)

Ran 5 tests for src/positionManager/test/PositionManager.t.sol:PositionManagerTest_module
[PASS] test_prepareCondition() (gas: 24726)
[PASS] test_removeModule() (gas: 29720)
[PASS] test_resolveCondition() (gas: 13771)
[PASS] test_revert_removeModule_unauthorized() (gas: 22951)
[PASS] test_revert_split_moduleNotRegistered() (gas: 24369)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 690.29µs (68.58µs CPU time)

Ran 4 tests for src/positionManager/test/PositionManager.t.sol:PositionManagerTest_moduleAuth
[PASS] test_revert_batchBurn_unauthorized() (gas: 27231)
[PASS] test_revert_batchMint_unauthorized() (gas: 27477)
[PASS] test_revert_burn_unauthorized() (gas: 25608)
[PASS] test_revert_mint_unauthorized() (gas: 25900)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 670.67µs (56.42µs CPU time)

Ran 1 test for src/oracle/test/OracleAggregator.t.sol:OracleAggregatorTest_initialize
[PASS] test_initializeBinaryEvent() (gas: 395030)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.98ms (81.29µs CPU time)

Ran 11 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_bridge
[PASS] test_burnFromBridge() (gas: 81631)
[PASS] test_burnFromBridge_batch() (gas: 131056)
[PASS] test_mintFromBridge() (gas: 73916)
[PASS] test_mintFromBridge_thenRedeem() (gas: 257252)
[PASS] test_mintFromBridge_zeroAmount() (gas: 37933)
[PASS] test_onERC1155BatchReceived_normalTransfer() (gas: 120472)
[PASS] test_onERC1155Received_normalTransfer() (gas: 76253)
[PASS] test_revert_burnFromBridge_wrongModuleId() (gas: 150659)
[PASS] test_revert_mintFromBridge_unauthorized() (gas: 23691)
[PASS] test_revert_mintFromBridge_wrongModuleId() (gas: 22210)
[PASS] test_revert_redeemBridgedPosition_unprepared() (gas: 152591)
Suite result: ok. 11 passed; 0 failed; 0 skipped; finished in 2.61ms (455.13µs CPU time)

Ran 1 test for src/positionManager/test/PositionManager.t.sol:PositionManagerTest_redeem
[PASS] test_redeem() (gas: 88033)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 648.67µs (38.42µs CPU time)

Ran 10 tests for src/positionManager/test/PositionManager.t.sol:PositionManagerTest_roles
[PASS] test_addAdmin() (gas: 50048)
[PASS] test_addCreator() (gas: 50153)
[PASS] test_addOperator() (gas: 50100)
[PASS] test_removeAdmin() (gas: 42030)
[PASS] test_removeCreator() (gas: 38867)
[PASS] test_removeOperator() (gas: 38871)
[PASS] test_revert_addAdmin_byAdmin() (gas: 27923)
[PASS] test_revert_addAdmin_unauthorized() (gas: 27889)
[PASS] test_revert_addCreator_unauthorized() (gas: 28021)
[PASS] test_revert_addOperator_unauthorized() (gas: 27924)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 783.96µs (155.17µs CPU time)

Ran 5 tests for src/positionManager/test/PositionManager.t.sol:PositionManagerTest_setCrossModuleAuth
[PASS] test_revert_setCrossModuleAuth_moduleIdZero() (gas: 280964)
[PASS] test_revert_setCrossModuleAuth_notRegistered() (gas: 583045)
[PASS] test_revert_setCrossModuleAuth_unauthorized() (gas: 19978)
[PASS] test_setCrossModuleAuth() (gas: 53772)
[PASS] test_setCrossModuleAuth_revoke() (gas: 42324)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 730.38µs (89.50µs CPU time)

Ran 3 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_canonicalization
[PASS] test_permutationYieldsSameId() (gas: 24885)
[PASS] test_revert_invalidModule() (gas: 14415)
[PASS] test_revert_invalidOutcomeIndex() (gas: 16602)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 2.06ms (50.29µs CPU time)

Ran 1 test for src/positionManager/test/PositionManager.t.sol:PositionManagerTest_split
[PASS] test_split() (gas: 84784)
```

```
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 633.63µs (28.42µs CPU time)

Ran 5 tests for src/positionManager/test/PositionManager.t.sol:PositionManagerTest_unsafeMint
[PASS] test_batchMintToNonReceiver() (gas: 120054)
[PASS] test_mintToNonReceiver() (gas: 90543)
[PASS] test_revert_batchMint_lengthMismatch() (gas: 26495)
[PASS] test_revert_batchMint_toZeroAddress() (gas: 24613)
[PASS] test_revert_mint_toZeroAddress() (gas: 20705)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 722.88µs (104.75µs CPU time)

Ran 2 tests for src/positionManager/test/PositionManager.t.sol:PositionManagerTest_upgrade
[PASS] test_revert_upgrade_unauthorized() (gas: 2129654)
[PASS] test_upgradeToAndCall() (gas: 2133890)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 698.04µs (97.54µs CPU time)

Ran 2 tests for src/positionManager/test/PositionManager.t.sol:PositionManagerTest_uri
[PASS] test_revert_uri_moduleNotRegistered() (gas: 17564)
[PASS] test_uri_returnsTemplate() (gas: 21955)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 625.42µs (26.21µs CPU time)

Ran 1 test for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_getLegs
[PASS] test_getLegs() (gas: 228851)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 7.13ms (190.00µs CPU time)

Ran 12 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_compress
[PASS] test_compress_fractionalDustCannotExtractAfterRemainingLegResolves() (gas: 604428)
[PASS] test_compress_fractionalDustDoesNotLeakCollateral() (gas: 548317)
[PASS] test_compress_noPosition_allTrueWorthless() (gas: 364172)
[PASS] test_compress_noPosition_falseConditionMakesCollateral() (gas: 351928)
[PASS] test_compress_noPosition_fractionalConditionMintsCollateralAndReducedNo() (gas: 446836)
[PASS] test_compress_noPosition_trueConditionReduces() (gas: 380336)
[PASS] test_compress_yesPosition_allTrue_collateral() (gas: 404291)
[PASS] test_compress_yesPosition_falseConditionMakesWorthless() (gas: 318383)
[PASS] test_compress_yesPosition_fractionalConditionAfterUnresolvedScalesDown() (gas: 399795)
[PASS] test_compress_yesPosition_fractionalConditionScalesDown() (gas: 403357)
[PASS] test_compress_yesPosition_trueConditionRemoved() (gas: 381719)
[PASS] test_revert_nothingToCompress() (gas: 257726)
Suite result: ok. 12 passed; 0 failed; 0 skipped; finished in 3.47ms (1.40ms CPU time)

Ran 3 tests for src/positionManager/test/PositionManager.t.sol:PositionManagerTest_view
[PASS] test_balanceOf_conditionId() (gas: 81470)
[PASS] test_getPayout() (gas: 25320)
[PASS] test_getPositionId() (gas: 12251)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 661.46µs (50.83µs CPU time)

Ran 1 test for src/routers/test/CtfRouter.t.sol:CtfRouterTest_splitPosition
[PASS] test_split() (gas: 300502)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.86ms (68.83µs CPU time)

Ran 14 tests for src/exchange/test/MatchOrders.t.sol:MatchOrdersTest_reverts
[PASS] test_revert_invalidComplement_makerWrongCondition() (gas: 366351)
[PASS] test_revert_invalidSignature() (gas: 448323)
[PASS] test_revert_invalidSignature_eoaSignerMakerMismatch() (gas: 436344)
[PASS] test_revert_invalidTokenId_takerNonBinaryPosition() (gas: 430768)
[PASS] test_revert_makingGtRemaining() (gas: 437722)
[PASS] test_revert_mismatchedArrayLengths() (gas: 431647)
[PASS] test_revert_noMakerOrders() (gas: 201848)
[PASS] test_revert_notOperator() (gas: 30991)
[PASS] test_revert_orderAlreadyFilled() (gas: 539980)
[PASS] test_revert_paused() (gas: 450500)
[PASS] test_revert_takerReceiveAmountMismatch() (gas: 510066)
[PASS] test_revert_userIsPaused() (gas: 459136)
[PASS] test_revert_zeroMakerAmount() (gas: 61252)
[PASS] test_revert_zeroMakerAmount_taker() (gas: 330548)
Suite result: ok. 14 passed; 0 failed; 0 skipped; finished in 12.37ms (10.11ms CPU time)

Ran 1 test for src/exchange/test/Preapproved.t.sol:PreapprovedTest_erc1271
[PASS] test_signerInvalidated() (gas: 783098)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.78ms (526.13µs CPU time)

Ran 2 tests for src/exchange/test/EIP712Digest.t.sol:Exchange_EIP712DigestStability
[PASS] test_orderHash_matchesIndependentComputation() (gas: 14030)
```

```
[PASS] test_orderStructHash_baseline() (gas: 1740)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.37ms (29.08µs CPU time)

Ran 1 test for
src/abstract/test/ERC1155TokenReceiver.t.sol:ERC1155TokenReceiverTest_onERC1155BatchReceived
[PASS] test_returnsAcceptanceSelector() (gas: 7927)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 71.75µs (11.88µs CPU time)

Ran 1 test for src/abstract/test/ERC1155TokenReceiver.t.sol:ERC1155TokenReceiverTest_onERC1155Received
[PASS] test_returnsAcceptanceSelector() (gas: 6370)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 62.96µs (6.92µs CPU time)

Ran 3 tests for src/abstract/test/ERC1155TokenReceiver.t.sol:ERC1155TokenReceiverTest_supportsInterface
[PASS] test_rejectsUnknown() (gas: 6468)
[PASS] test_supportsERC1155TokenReceiver() (gas: 5588)
[PASS] test_supportsERC165() (gas: 5617)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 71.79µs (16.87µs CPU time)

Ran 2 tests for src/exchange/test/MatchOrders.t.sol:MatchOrdersTest_validateOrderBranches
[PASS] test_revert_validateOrder_alreadyFilled() (gas: 535274)
[PASS] test_revert_validateOrder_userPaused() (gas: 57514)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.64ms (421.87µs CPU time)

Ran 2 tests for src/routers/test/RouterBinaryModule.t.sol:RouterBinaryModuleTest_split
[PASS] test_split() (gas: 297922)
[PASS] test_split_anyCondition() (gas: 278472)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 4.24ms (133.21µs CPU time)

Ran 10 tests for src/modules/test/MigrationSnapshots.t.sol:MigrationSnapshots_Test
[PASS] test_MigrationSnapshots_binary_migrate_batch16() (gas: 747611)
[PASS] test_MigrationSnapshots_binary_migrate_batch2() (gas: 438560)
[PASS] test_MigrationSnapshots_binary_migrate_batch4() (gas: 478908)
[PASS] test_MigrationSnapshots_binary_migrate_batch8() (gas: 559705)
[PASS] test_MigrationSnapshots_binary_migrate_operator_batch4() (gas: 482280)
[PASS] test_MigrationSnapshots_negRisk_migrate_batch16() (gas: 5650625)
[PASS] test_MigrationSnapshots_negRisk_migrate_batch2() (gas: 1044495)
[PASS] test_MigrationSnapshots_negRisk_migrate_batch4() (gas: 1695188)
[PASS] test_MigrationSnapshots_negRisk_migrate_batch8() (gas: 3030984)
[PASS] test_MigrationSnapshots_negRisk_migrate_operator_batch4() (gas: 1698526)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 42.17ms (32.34ms CPU time)

Ran 12 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_getPayout
[PASS] test_getPayout_noAllTrue() (gas: 336204)
[PASS] test_getPayout_noAnyFalse() (gas: 335989)
[PASS] test_getPayout_noFractionalComplement() (gas: 376336)
[PASS] test_getPayout_noLaterFalseBeforeAllResolved() (gas: 271743)
[PASS] test_getPayout_supportsManyFullPayoutLegs() (gas: 1974307)
[PASS] test_getPayout_supportsThirteenFractionalLegs() (gas: 1556813)
[PASS] test_getPayout_yesAllTrue() (gas: 336118)
[PASS] test_getPayout_yesAnyFalse() (gas: 331736)
[PASS] test_getPayout_yesFractionalProduct() (gas: 376219)
[PASS] test_getPayout_yesFractionalRoundsAtEnd() (gas: 372179)
[PASS] test_getPayout_yesLaterFalseBeforeAllResolved() (gas: 271786)
[PASS] test_revert_unresolved() (gas: 277391)
Suite result: ok. 12 passed; 0 failed; 0 skipped; finished in 6.93ms (1.85ms CPU time)

Ran 4 tests for src/libraries/test/Ids.t.sol:IdsTest_canonicalization
[PASS] test_revert_conditionIdNonCanonical_outcomeByte(bytes32,uint8) (runs: 256, µ: 9209, ~: 9209)
[PASS] test_revert_eventIdNonCanonical_bothDirty(bytes32,uint8,uint16) (runs: 256, µ: 9961, ~: 9961)
[PASS] test_revert_eventIdNonCanonical_conditionIndex(bytes32,uint16) (runs: 256, µ: 11247, ~: 11247)
[PASS] test_revert_eventIdNonCanonical_outcomeByte(bytes32,uint8) (runs: 256, µ: 9490, ~: 9490)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 15.69ms (15.39ms CPU time)

Ran 1 test for src/exchange/test/ERC1271Signature.t.sol:ERC1271SignatureTest_gasGriefing
[PASS] test_validSignature_boundsReturndata() (gas: 2961578)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.81ms (435.13µs CPU time)

Ran 1 test for src/libraries/test/Ids.t.sol:EventIdLibTest_conditionRoundtrip
[PASS] test_eventConditionRoundtrip(bytes32,uint256) (runs: 256, µ: 902, ~: 902)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 21.42ms (21.14ms CPU time)

Ran 1 test for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_initialize
```

```
[PASS] test_revert_cannotReinitialize() (gas: 22600)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.01ms (9.71µs CPU time)

Ran 2 tests for
src/modules/test/NegRiskMigration.t.sol:NegRiskMigrationTest_resolveConditionToNo_migration
[PASS] test_resolveConditionToNo_migratedLoser_redeemsLegacy() (gas: 1939557)
[PASS] test_revert_resolveConditionToNo_migratedLoser_legacyUnresolved() (gas: 1484494)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 12.59ms (10.71ms CPU time)

Ran 2 tests for src/exchange/test/MatchOrdersBalanceFlows.t.sol:MatchOrdersBalanceFlowsTest_merge
[PASS] test_twoMakers() (gas: 1010525)
[PASS] test_twoMakersWithFees() (gas: 1038241)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 3.45ms (1.21ms CPU time)

Ran 3 tests for src/exchange/test/ERC1271Signature.t.sol:ERC1271SignatureTest_reverts
[PASS] test_revert_incorrectSigner() (gas: 443521)
[PASS] test_revert_invalidSignerMaker() (gas: 439694)
[PASS] test_revert_nonContract() (gas: 438173)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 2.94ms (729.21µs CPU time)

Ran 3 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_inject
[PASS] test_inject() (gas: 534656)
[PASS] test_revert_inject_requiresNoSide() (gas: 217035)
[PASS] test_revert_inject_unprepared() (gas: 144048)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 2.28ms (252.92µs CPU time)

Ran 6 tests for src/collateral/test/CollateralToken.t.sol:CollateralTokenTest_unwrap
[PASS] test_revert_invalidAsset(address) (runs: 256, µ: 27096, ~: 27096)
[PASS] test_revert_unauthorized() (gas: 24300)
[PASS] test_revert_unwrap_noPreTransfer() (gas: 105440)
[PASS] test_unwrapUSDC() (gas: 170440)
[PASS] test_unwrapUSDC_pretransfer() (gas: 166987)
[PASS] test_unwrapUSDCe() (gas: 172069)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 28.95ms (29.11ms CPU time)

Ran 8 tests for src/modules/test/NegRiskMigration.t.sol:NegRiskMigrationTest_prepareEvent
[PASS] test_maxLegacyConditionCount() (gas: 6219530)
[PASS] test_minLegacyConditionCount() (gas: 94933)
[PASS] test_prepareMigrationEvent_emitsLegacyEventIdInEventPrepared() (gas: 1535899)
[PASS] test_revert_alreadyPrepared() (gas: 1536513)
[PASS] test_revert_eventNotPrepared() (gas: 23320)
[PASS] test_revert_insufficientConditionCount() (gas: 21324)
[PASS] test_revert_legacyConditionCountTooLarge() (gas: 21309)
[PASS] test_revert_migrationNotSupported() (gas: 5618342)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 18.28ms (16.41ms CPU time)

Ran 1 test for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_view
[PASS] test_getPositionId(bytes32,uint256) (runs: 256, µ: 393, ~: 393)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 30.37ms (25.62ms CPU time)

Ran 1 test for src/exchange/test/ERC1271Signature.t.sol:ERC1271SignatureTest_valid
[PASS] test_validSignature() (gas: 537611)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.52ms (306.42µs CPU time)

Ran 5 tests for src/exchange/test/Preapproved.t.sol:PreapprovedTest_matching
[PASS] test_partialFill() (gas: 850752)
[PASS] test_preapprovedMakerComplementary() (gas: 559492)
[PASS] test_preapprovedTakerComplementary() (gas: 559487)
[PASS] test_respectsFilledStatus() (gas: 583087)
[PASS] test_respectsUserPause() (gas: 484336)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 9.73ms (6.12ms CPU time)

Ran 2 tests for src/exchange/test/MatchOrdersBalanceFlows.t.sol:MatchOrdersBalanceFlowsTest_mint
[PASS] test_twoMakers() (gas: 704305)
[PASS] test_twoMakersWithFees() (gas: 731796)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 3.25ms (1.07ms CPU time)

Ran 6 tests for src/modules/test/NegRiskMigration.t.sol:NegRiskMigrationTest_resolveMigrationCondition
[PASS] test_resolveMigrationCondition() (gas: 20584944)
[PASS] test_revert_alreadyResolved() (gas: 1959120)
[PASS] test_revert_nonMigratedCondition() (gas: 1440015)
[PASS] test_revert_notResolvedOnLegacy() (gas: 1541112)
```

```
[PASS] test_revert_resolutionPaused() (gas: 1604027)
[PASS] test_unpauseAllowsResolution() (gas: 1817399)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 51.63ms (46.84ms CPU time)

Ran 3 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_merge
[PASS] test_merge() (gas: 315892)
[PASS] test_revert_merge_wrongModuleId() (gas: 17708)
[PASS] test_splitMerge_roundtrip() (gas: 303408)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 2.54ms (529.88µs CPU time)

Ran 3 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_mergeFromYesBasket
[PASS] test_mergeFromYesBasket() (gas: 622716)
[PASS] test_revert_mergeFromYesBasket_requiresNoSide() (gas: 217016)
[PASS] test_revert_mergeFromYesBasket_unprepared() (gas: 144028)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 2.35ms (285.08µs CPU time)

Ran 1 test for src/modules/test/NegRiskMigration.t.sol:NegRiskMigrationTest_horizontalSplit
[PASS] test_horizontalSplit() (gas: 21589800)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 28.75ms (26.89ms CPU time)

Ran 2 tests for src/libraries/test/Ids.t.sol:IdsTest_conditionEq
[PASS] test_conditionEq_distinct(bytes32,bytes32) (runs: 256, µ: 3491, ~: 3491)
[PASS] test_conditionEq_reflexive(bytes32) (runs: 256, µ: 480, ~: 480)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 5.90ms (5.61ms CPU time)

Ran 1 test for src/exchange/test/ExchangeAdmin.t.sol:ExchangeAdminTest_domainSeparator
[PASS] test_domainSeparator() (gas: 10560)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.19ms (9.79µs CPU time)

Ran 4 tests for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_convert
[PASS] test_convert() (gas: 375756)
[PASS] test_convert_emitsPositionConverted() (gas: 377310)
[PASS] test_revert_invalidConditionIndex() (gas: 20958)
[PASS] test_revert_invalidEventId() (gas: 16718)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 2.45ms (551.67µs CPU time)

Ran 2 tests for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_getEventId
[PASS] test_getEventId() (gas: 13313)
[PASS] test_revert_invalidConditionCount() (gas: 28914)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 1.88ms (34.83µs CPU time)

Ran 2 tests for src/exchange/test/ExchangeAdmin.t.sol:ExchangeAdminTest_hashOrder
[PASS] test_deterministic() (gas: 19684)
[PASS] test_differentSaltDifferentHash() (gas: 20579)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.23ms (43.58µs CPU time)

Ran 5 tests for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_directHorizontal
[PASS] test_horizontalMerge_direct() (gas: 446412)
[PASS] test_horizontalSplit_direct() (gas: 266353)
[PASS] test_revert_convert_noPreTransfer() (gas: 111915)
[PASS] test_revert_horizontalMerge_noPreTransfer() (gas: 93754)
[PASS] test_revert_horizontalSplit_noPreTransfer() (gas: 141458)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 2.16ms (258.92µs CPU time)

Ran 3 tests for src/exchange/test/Preapproved.t.sol:PreapprovedTest_preapproval
[PASS] test_invalidatePreapprovedOrder() (gas: 48854)
[PASS] test_preapproveOrder_thenMatch() (gas: 586329)
[PASS] test_revert_preapproveOrder_notOperator() (gas: 26959)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 4.69ms (610.17µs CPU time)

Ran 6 tests for src/exchange/test/MatchOrdersBalanceFlows.t.sol:MatchOrdersBalanceFlowsTest_mixedPath
[PASS] test_comboComplementaryMerge() (gas: 889588)
[PASS] test_comboComplementaryMergeWithFees() (gas: 917243)
[PASS] test_comboComplementaryMintNoFees() (gas: 800792)
[PASS] test_comboComplementaryMintWithFees() (gas: 5341288)
[PASS] test_threeMakersCompMergeWithFees() (gas: 1175976)
[PASS] test_threeMakersCompMintWithFees() (gas: 997617)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 6.12ms (3.87ms CPU time)

Ran 3 tests for src/modules/test/NegRiskMigration.t.sol:NegRiskMigrationTest_horizontalMerge
[PASS] test_horizontalMerge() (gas: 23324660)
[PASS] test_horizontalMerge_beforeResolution() (gas: 1341742)
```

```
[PASS] test_horizontalMerge_nonMigratedEvent() (gas: 427205)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 27.87ms (27.71ms CPU time)

Ran 9 tests for src/positionManager/test/UnsafeTransfer.t.sol:UnsafeTransferTest_unsafeTransferFrom
[PASS] test_approved() (gas: 80775)
[PASS] test_basic() (gas: 52817)
[PASS] test_emitsTransferSingle() (gas: 52805)
[PASS] test_revert_insufficientBalance() (gas: 22408)
[PASS] test_revert_notApproved() (gas: 22388)
[PASS] test_revert_toZeroAddress() (gas: 18130)
[PASS] test_selfTransfer() (gas: 24420)
[PASS] test_toNonReceiver() (gas: 93614)
[PASS] test_zeroAmount() (gas: 30094)
Suite result: ok. 9 passed; 0 failed; 0 skipped; finished in 33.71ms (138.79µs CPU time)

Ran 3 tests for src/exchange/test/ExchangeAdmin.t.sol:ExchangeAdminTest_initializer
[PASS] test_revert_constructor_maxFeeRateExceedsCeiling() (gas: 59413)
[PASS] test_revert_initialize_onImplementation() (gas: 4526219)
[PASS] test_revert_initialize_twice() (gas: 22656)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 2.35ms (163.00µs CPU time)

Ran 4 tests for src/exchange/test/Preapproved.t.sol:PreapprovedTest_reverts
[PASS] test_revert_invalidSignature() (gas: 34470)
[PASS] test_revert_invalidSignatureNotPreapproved() (gas: 439623)
[PASS] test_revert_invalidatedPreapproval() (gas: 479240)
[PASS] test_revert_notOperator() (gas: 25688)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 3.01ms (765.54µs CPU time)

Ran 4 tests for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_getPayout
[PASS] test_fullPayout() (gas: 203806)
[PASS] test_partialPayout() (gas: 170297)
[PASS] test_revert_conditionNotResolved() (gas: 40480)
[PASS] test_revert_invalidOutcomeIndex() (gas: 200767)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 5.92ms (135.71µs CPU time)

Ran 1 test for src/exchange/test/ExchangeAdmin.t.sol:ExchangeAdminTest_operators
[PASS] test_addRemoveOperator() (gas: 35128)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 2.57ms (44.88µs CPU time)

Ran 2 tests for src/exchange/test/ExchangeMath.t.sol:ExchangeMathTest
[PASS] test_calculateTakingAmount_fuzz(uint64,uint128,uint128) (runs: 256, µ: 9325, ~: 9325)
[PASS]
test_complementaryAggregateCrossingInvariant_fuzz(uint64,uint64,uint64,uint64,uint64,uint64,uint64,uint64
) (runs: 256, µ: 14028, ~: 14028)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 38.04ms (67.38ms CPU time)

Ran 8 tests for src/exchange/test/MatchOrdersBalanceFlows.t.sol:MatchOrdersBalanceFlowsTest_complementary
[PASS] test_complementaryBuyMakerFeeOnly() (gas: 569512)
[PASS] test_complementaryBuyTakerFeeOnly() (gas: 569238)
[PASS] test_complementarySellMakerFeeOnly() (gas: 603678)
[PASS] test_complementarySellMixedMakerFees() (gas: 770492)
[PASS] test_takerBuyTwoMakers() (gas: 800368)
[PASS] test_takerBuyTwoMakersWithFees() (gas: 828747)
[PASS] test_takerSellTwoMakers() (gas: 726593)
[PASS] test_takerSellTwoMakersWithFees() (gas: 773199)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 20.08ms (16.83ms CPU time)

Ran 2 tests for src/exchange/test/MatchOrders.t.sol:MatchOrdersTest_batchMatchTakerBuy
[PASS] test_revert_batchTakerBuy_makerBuyComplement_notCrossing() (gas: 597236)
[PASS] test_revert_batchTakerBuy_makerSell_notCrossing() (gas: 596696)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 3.03ms (762.08µs CPU time)

Ran 2 tests for
src/exchange/test/ExchangeSnapshots.t.sol:ExchangeSnapshotsTest_comboComplementaryMergeFees
[PASS] test_tenMakers() (gas: 3021744)
[PASS] test_twentyMakers() (gas: 5567548)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 13.34ms (8.61ms CPU time)

Ran 4 tests for src/exchange/test/ExchangeAdmin.t.sol:ExchangeAdminTest_pause
[PASS] test_pauseThenMatchReverts() (gas: 558509)
[PASS] test_pauseUnpause() (gas: 32607)
[PASS] test_revert_pauseUser_alreadyPaused() (gas: 43039)
```

```
[PASS] test_selfServicePauseUser() (gas: 38092)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 6.26ms (1.13ms CPU time)

Ran 3 tests for src/exchange/test/MatchOrders.t.sol:MatchOrdersTest_batchMatchTakerSell
[PASS] test_revert_batchTakerSell_makerBuy_notCrossing() (gas: 682951)
[PASS] test_revert_batchTakerSell_makerSellComplement_notCrossing() (gas: 698715)
[PASS] test_revert_batchTakerSell_takerFillMismatch() (gas: 783556)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 3.55ms (1.28ms CPU time)

Ran 4 tests for src/libraries/test/Ids.t.sol:IdsTest_encodeIdempotence
[PASS] test_conditionIdEncodeFromDataIdempotent(uint256,uint256,bytes) (runs: 256, μ: 1472, ~: 1450)
[PASS] test_conditionIdEncodeFromIdempotent(uint256,bytes32,uint256,uint256) (runs: 256, μ: 690, ~: 690)
[PASS] test_eventIdEncodeFromDataIdempotent(uint256,uint256,bytes) (runs: 256, μ: 1642, ~: 1627)
[PASS] test_eventIdEncodeFromIdempotent(uint256,bytes32,uint256) (runs: 256, μ: 872, ~: 872)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 15.77ms (15.48ms CPU time)

Ran 2 tests for src/modules/test/NegRiskMigration.t.sol:NegRiskMigrationTest_view
[PASS] test_getConditionId(bytes32,uint8) (runs: 256, μ: 3176, ~: 3176)
[PASS] test_positionIds(bytes32,uint8,bool) (runs: 256, μ: 953, ~: 964)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 10.99ms (15.99ms CPU time)

Ran 2 tests for src/exchange/test/ExchangeSnapshots.t.sol:ExchangeSnapshotsTest_comboComplementaryMerge
[PASS] test_tenMakers() (gas: 2565412)
[PASS] test_twentyMakers() (gas: 4676295)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 7.91ms (5.80ms CPU time)

Ran 6 tests for src/exchange/test/MatchOrdersBalanceFlows.t.sol:MatchOrdersBalanceFlowsTest_surplusRefund
[PASS] test_batchBuyCollateralSurplusRefunded() (gas: 555424)
[PASS] test_batchBuyPositionSurplusRefunded() (gas: 822985)
[PASS] test_batchBuyPositionSurplusWithFees() (gas: 846703)
[PASS] test_batchMintOverfundedRefundedToTaker() (gas: 555426)
[PASS] test_batchSellPositiveSlippageRefundedToTaker() (gas: 892933)
[PASS] test_batchSellPositiveSlippageWithFees() (gas: 916232)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 11.75ms (3.02ms CPU time)

Ran 3 tests for src/routers/test/RouterNegRiskModule.t.sol:RouterNegRiskModule_Test_events
[PASS] test_convert_emitsPositionConverted() (gas: 7387289)
[PASS] test_horizontalMerge_emitsHorizontalMerge() (gas: 10349715)
[PASS] test_horizontalSplit_emitsHorizontalSplit() (gas: 6219947)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 7.15ms (5.24ms CPU time)

Ran 3 tests for src/libraries/test/Ids.t.sol:PositionIdLibTest_pathEquivalence
[PASS] test_conditionIdPaths(uint256,uint256,bytes) (runs: 256, μ: 2233, ~: 2185)
[PASS] test_eventIdPaths(uint256,uint256,bytes) (runs: 256, μ: 2155, ~: 2122)
[PASS] test_positionIdPaths(uint256,uint256,uint256,bytes) (runs: 256, μ: 2301, ~: 2271)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 11.16ms (10.87ms CPU time)

Ran 2 tests for src/libraries/test/Ids.t.sol:IdsTest_positionEq
[PASS] test_positionEq_distinct(uint256,uint256) (runs: 256, μ: 3293, ~: 3293)
[PASS] test_positionEq_reflexive(uint256) (runs: 256, μ: 388, ~: 388)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 15.35ms (17.68ms CPU time)

Ran 12 tests for src/modules/test/NegRiskModule.t.sol:NegRiskModuleTest_bridge
[PASS] test_burnFromBridge() (gas: 115384)
[PASS] test_burnFromBridge_batch() (gas: 165778)
[PASS] test_conditionCount() (gas: 13888)
[PASS] test_conditionCount_invalidEventId() (gas: 11060)
[PASS] test_mintFromBridge() (gas: 108681)
[PASS] test_mintFromBridge_thenRedeem() (gas: 362027)
[PASS] test_mintFromBridge_zeroAmount() (gas: 80744)
[PASS] test_onERC1155BatchReceived_normalTransfer() (gas: 160301)
[PASS] test_onERC1155Received_normalTransfer() (gas: 119229)
[PASS] test_reportResult_bridgeDoesNotRequireEventBridge() (gas: 181430)
[PASS] test_reportResult_bridgeIdempotent() (gas: 184343)
[PASS] test_revert_mintFromBridge_unauthorized() (gas: 65096)
Suite result: ok. 12 passed; 0 failed; 0 skipped; finished in 2.41ms (462.71µs CPU time)

Ran 2 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_wrap
[PASS] test_wrap() (gas: 239929)
[PASS] test_wrapUnwrap_roundtrip() (gas: 287188)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 2.16ms (146.33µs CPU time)
```

```
Ran 5 tests for src/exchange/test/MatchOrders.t.sol:MatchOrdersTest_complementValidation
[PASS] test_revert_invalidComplement_sameOutcome() (gas: 354956)
[PASS] test_revert_mismatchedTokenIds_batchSellBuyMaker() (gas: 460268)
[PASS] test_revert_notCrossing_batchBuySellMaker() (gas: 441471)
[PASS] test_revert_notCrossing_mergePath() (gas: 551976)
[PASS] test_revert_notCrossing_mintPath() (gas: 356948)
Suite result: ok. 5 passed; 0 failed; 0 skipped; finished in 3.63ms (1.38ms CPU time)

Ran 2 tests for src/exchange/test/ExchangeSnapshots.t.sol:ExchangeSnapshotsTest_comboComplementaryMint
[PASS] test_tenMakers() (gas: 2477894)
[PASS] test_twentyMakers() (gas: 4586258)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 8.03ms (5.96ms CPU time)

Ran 2 tests for src/libraries/test/Ids.t.sol:PositionIdLibTest_structuralInvariants
[PASS] test_conditionIdOutcomeByteZero(uint256,uint256,bytes) (runs: 256, μ: 1424, ~: 1420)
[PASS] test_reservedBitsZero(uint256,uint256,uint256,bytes) (runs: 256, μ: 1428, ~: 1408)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 10.49ms (10.20ms CPU time)

Ran 2 tests for src/routers/test/RouterNegRiskModule.t.sol:RouterNegRiskModule_Test_horizontal
[PASS] test_horizontalMerge() (gas: 16400518)
[PASS] test_horizontalSplit() (gas: 7633306)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 5.59ms (5.40ms CPU time)

Ran 8 tests for src/exchange/test/MatchOrders.t.sol:MatchOrdersTest_complementary
[PASS] test_complementary() (gas: 556640)
[PASS] test_complementaryConsumesLimitFill_whenPriceImproves() (gas: 550811)
[PASS] test_complementaryFees() (gas: 564481)
[PASS] test_matchOrdersAndPrepareCombinatorial() (gas: 589416)
[PASS] test_noExchangeBalance() (gas: 555074)
[PASS] test_revert_complementaryFillExceedsTakerFill_overspend() (gas: 510669)
[PASS] test_revert_matchOrdersAndPrepareCombinatorial_notOperator() (gas: 20243)
[PASS] test_revert_matchOrdersAndPrepareCombinatorial_paused() (gas: 46009)
Suite result: ok. 8 passed; 0 failed; 0 skipped; finished in 5.26ms (3.04ms CPU time)

Ran 3 tests for src/exchange/test/ExchangeAdmin.t.sol:ExchangeAdminTest_removeAdminAndOperator
[PASS] test_removeAdmin() (gas: 35024)
[PASS] test_revert_removeAdmin_unauthorized() (gas: 46575)
[PASS] test_revert_removeOperator_unauthorized() (gas: 48648)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 7.50ms (545.50µs CPU time)

Ran 1 test for src/libraries/test/Ids.t.sol:PositionIdLibTest_roundtrip
[PASS] test_positionId(uint256,uint256,uint256,bytes) (runs: 256, μ: 1541, ~: 1524)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 12.41ms (12.12ms CPU time)

Ran 12 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_split
[PASS] test_maxLegs() (gas: 10266)
[PASS] test_prepareCondition_maxLegs() (gas: 1635507)
[PASS] test_revert_conflictingConditions() (gas: 19828)
[PASS] test_revert_duplicateCondition() (gas: 17503)
[PASS] test_revert_emptyConditions() (gas: 13828)
[PASS] test_revert_nonCanonicalOrder() (gas: 19752)
[PASS] test_revert_prepareCondition_tooManyLegs() (gas: 431702)
[PASS] test_revert_splitOnCondition_tooManyLegs() (gas: 1661486)
[PASS] test_revert_split_wrongModuleId() (gas: 18648)
[PASS] test_revert_wrongToLength() (gas: 21676)
[PASS] test_split() (gas: 150930)
[PASS] test_split_singleCondition() (gas: 177737)
Suite result: ok. 12 passed; 0 failed; 0 skipped; finished in 4.54ms (2.51ms CPU time)

Ran 4 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_unwrap
[PASS] test_revert_unwrap_multiCondition() (gas: 241390)
[PASS] test_unwrap_noPosition() (gas: 253661)
[PASS] test_unwrap_yesPosition() (gas: 251958)
[PASS] test_unwrap_yesPosition_ofNCondition() (gas: 250791)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 2.31ms (259.00µs CPU time)

Ran 6 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_splitOnCondition
[PASS] test_revert_conditionAlreadyPresent() (gas: 199135)
[PASS] test_revert_splitOnCondition_parentMustBeYes() (gas: 198298)
[PASS] test_revert_splitOnCondition_parentUnprepared() (gas: 143863)
[PASS] test_revert_splitOnCondition_referenceMustBeConditionId() (gas: 6312)
[PASS] test_revert_splitOnCondition_wrongToLength() (gas: 243193)
```

```
[PASS] test_splitOnCondition() (gas: 478781)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 2.45ms (345.29µs CPU time)

Ran 4 tests for src/routers/test/CtfRouter.t.sol:CtfRouterTest_initialize
[PASS] test_initialize_setsOwner() (gas: 14532)
[PASS] test_revert_initialize_onImplementation() (gas: 1253369)
[PASS] test_revert_initialize_twice() (gas: 20470)
[PASS] test_revert_initialize_zeroOwner() (gas: 1340276)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 2.00ms (85.92µs CPU time)

Ran 1 test for src/modules/test/CombinatorialModuleInvariant.t.sol:CombinatorialModuleInvariantTest
[PASS] test_handlerResolvePath() (gas: 158795)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 6.58ms (49.58µs CPU time)

Ran 2 tests for src/exchange/test/ExchangeAdmin.t.sol:ExchangeAdminTest_renounceOperatorRole
[PASS] test_renounceOperatorRole() (gas: 20242)
[PASS] test_revert_renounceOperatorRole_notOperator() (gas: 16840)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 5.17ms (156.88µs CPU time)

Ran 1 test for src/routers/test/CtfRouter.t.sol:CtfRouterTest_mergePositions
[PASS] test_merge() (gas: 415664)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 1.93ms (106.29µs CPU time)

Ran 4 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_upgrade
[PASS] test_revert_upgrade_incompatibleModuleId() (gas: 4446854)
[PASS] test_revert_upgrade_incompatiblePositionManager() (gas: 30406969)
[PASS] test_revert_upgrade_unauthorized() (gas: 5016029)
[PASS] test_upgradeToAndCall_preservesState() (gas: 5104712)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 3.56ms (1.55ms CPU time)

Ran 2 tests for src/libraries/test/Ids.t.sol:ConditionIdLibTest_isValidEventId
[PASS] test_isValidEventId_falseForSubcondition() (gas: 1148)
[PASS] test_isValidEventId_trueForEvent() (gas: 1039)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 299.54µs (14.25µs CPU time)

Ran 4 tests for src/exchange/test/ExchangeSnapshots.t.sol:ExchangeSnapshotsTest_merge
[PASS] test_fiveMakers() (gas: 1813101)
[PASS] test_oneMaker() (gas: 729503)
[PASS] test_tenMakers() (gas: 3170002)
[PASS] test_twentyMakers() (gas: 5885632)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 9.77ms (7.72ms CPU time)

Ran 2 tests for src/routers/test/CtfRouter.t.sol:CtfRouterTest_authorizeUpgrade
[PASS] test_revert_upgrade_unauthorized() (gas: 1265453)
[PASS] test_upgrade_ownerSucceeds() (gas: 26666957)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 3.05ms (1.21ms CPU time)

Ran 3 tests for src/routers/test/CtfRouter.t.sol:CtfRouterTest_redeemInvalidIndexSet
[PASS] test_revert_redeemPositions_indexSetInvalidAmongValid() (gas: 419351)
[PASS] test_revert_redeemPositions_indexSetThree() (gas: 38048)
[PASS] test_revert_redeemPositions_indexSetZero() (gas: 38069)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 2.01ms (144.13µs CPU time)

Ran 3 tests for src/routers/test/CtfRouter.t.sol:CtfRouterTest_events
[PASS] test_mergePositions_emitsPositionsMerged() (gas: 413905)
[PASS] test_redeemPositions_emitsPositionRedeemedPerIndex() (gas: 478498)
[PASS] test_splitPosition_emitsPositionSplit() (gas: 279158)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 2.78ms (1.32ms CPU time)

Ran 6 tests for src/modules/test/NegRiskMigration.t.sol:NegRiskMigrationTest_resolveMigrationCounters
[PASS] test_countersUpdateOnNo() (gas: 1730775)
[PASS] test_countersUpdateOnYes() (gas: 1792004)
[PASS] test_finalizeMigration_allNo() (gas: 1770665)
[PASS] test_migrationThenResolveConditionToNo() (gas: 1771930)
[PASS] test_revert_lastConditionMustBeNo() (gas: 1682390)
[PASS] test_revert_twoYesLegacyResolves() (gas: 1836295)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 31.43ms (32.04ms CPU time)

Ran 2 tests for src/exchange/test/ExchangeAdmin.t.sol:ExchangeAdminTest_roleViews
[PASS] test_isAdmin() (gas: 20383)
[PASS] test_isOperator() (gas: 27555)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 3.36ms (50.75µs CPU time)
```

```
Ran 3 tests for src/exchange/test/ExchangeSnapshots.t.sol:ExchangeSnapshotsTest_complementarySell
[PASS] test_oneMaker() (gas: 548839)
[PASS] test_tenMakers() (gas: 2031708)
[PASS] test_twentyMakers() (gas: 3683364)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 8.23ms (5.68ms CPU time)

Ran 4 tests for src/exchange/test/ExchangeSnapshots.t.sol:ExchangeSnapshotsTest_complementary
[PASS] test_fiveMakers() (gas: 1549219)
[PASS] test_oneMaker() (gas: 533028)
[PASS] test_tenMakers() (gas: 2821864)
[PASS] test_twentyMakers() (gas: 5368917)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 10.96ms (12.61ms CPU time)

Ran 4 tests for src/exchange/test/ExchangeSnapshots.t.sol:ExchangeSnapshotsTest_mint
[PASS] test_fiveMakers() (gas: 1190696)
[PASS] test_oneMaker() (gas: 528644)
[PASS] test_tenMakers() (gas: 2020665)
[PASS] test_twentyMakers() (gas: 3682368)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 6.42ms (7.58ms CPU time)

Ran 1 test for src/routers/test/RouterNegRiskModule.t.sol:RouterNegRiskModule_Test_split
[PASS] test_split(uint256) (runs: 256, μ: 279609, ~: 279617)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 23.63ms (21.74ms CPU time)

Ran 1 test for src/routers/test/RouterNegRiskModule.t.sol:RouterNegRiskModule_Test_merge
[PASS] test_merge(uint256) (runs: 256, μ: 395525, ~: 395540)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 48.46ms (45.38ms CPU time)

Ran 1 test for src/routers/test/CtfRouter.t.sol:CtfRouterTest_redeemPositions
[PASS] test_redeem(bool,uint256) (runs: 256, μ: 455354, ~: 470994)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 32.84ms (31.03ms CPU time)

Ran 4 tests for src/modules/test/NegRiskMigration.t.sol:NegRiskMigrationTest_migrate
[PASS] test_migrate() (gas: 20543645)
[PASS] test_revert_amountsLengthMismatch() (gas: 1537147)
[PASS] test_revert_conditionNotPrepared() (gas: 1438931)
[PASS] test_revert_invalidArrayLength() (gas: 1537165)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 61.88ms (32.59ms CPU time)

Ran 1 test for src/libraries/test/Ids.t.sol:ConditionIdLibTest_positionId
[PASS] test_conditionToPosition(bytes32,uint256) (runs: 256, μ: 553, ~: 553)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 34.23ms (33.94ms CPU time)

Ran 2 tests for src/libraries/test/Ids.t.sol:IdsTest_eventEq
[PASS] test_eventEq_distinct(bytes32,bytes32) (runs: 256, μ: 4286, ~: 4286)
[PASS] test_eventEq_reflexive(bytes32) (runs: 256, μ: 796, ~: 796)
Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 56.28ms (58.87ms CPU time)

Ran 1 test for src/libraries/test/Ids.t.sol:IdsTest_computeBaseHash
[PASS] test_computeBaseHash(uint256,bytes) (runs: 256, μ: 1390, ~: 1368)
Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 70.53ms (70.25ms CPU time)

Ran 10 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_pathIndependence
[PASS] testFuzz_extractPayoutProperty(uint96,uint256,uint256,uint256) (runs: 256, μ: 603723, ~: 606022)
[PASS] testFuzz_yesBasketPayoutProperty(uint96,uint256,uint256,uint256) (runs: 256, μ: 672950, ~: 674671)
[PASS] test_collateralReturnViaPartialPeel() (gas: 1203844)
[PASS] test_convertToYesBasket_matches_repeatedExtract() (gas: 771439)
[PASS] test_extractInjectRoundtrip() (gas: 533471)
[PASS] test_extractPayoutIdentity_boolean() (gas: 533220)
[PASS] test_extractPayoutWeakInequality_fractional() (gas: 598530)
[PASS] test_splitMergeValuePreservation() (gas: 621020)
[PASS] test_yesBasketPayoutIdentity_boolean() (gas: 597093)
[PASS] test_yesBasketPayoutWeakInequality_fractional() (gas: 667121)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 129.22ms (208.86ms CPU time)

Ran 10 tests for src/modules/test/CombinatorialModule.t.sol:CombinatorialModuleTest_redeem
[PASS] test_redeem_fractionalYesNoPairCannotExtractDust() (gas: 467364)
[PASS] test_redeem_noPosition_allTrue() (gas: 364204)
[PASS] test_redeem_noPosition_anyFalse() (gas: 404099)
[PASS] test_redeem_noPosition_fractionalComplement() (gas: 447593)
[PASS] test_redeem_noPosition_laterFalseBeforeAllResolved() (gas: 347288)
```

```
[PASS] test_redeem_yesPosition_allTrue() (gas: 407353)
[PASS] test_redeem_yesPosition_anyFalse() (gas: 360613)
[PASS] test_redeem_yesPosition_fractionalProduct() (gas: 447454)
[PASS] test_redeem_yesPosition_laterFalseBeforeAllResolved() (gas: 312632)
[PASS] test_revert_notFullyResolved() (gas: 314117)
Suite result: ok. 10 passed; 0 failed; 0 skipped; finished in 2.71ms (2.39ms CPU time)

Ran 3 tests for src/routers/test/RouterNegRiskModule.t.sol:RouterNegRiskModule_Test_convert
[PASS] test_convert(uint256) (runs: 256, μ: 7452812, ~: 7452812)
[PASS] test_revert_conditionIndexMaxUint16() (gas: 117125)
[PASS] test_revert_conditionIndexOutOfRange() (gas: 119444)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 346.51ms (344.69ms CPU time)

Ran 4 tests for src/routers/test/RouterBinaryModule.t.sol:RouterBinaryModuleTest_redeem
[PASS] test_redeem(bool) (runs: 256, μ: 524687, ~: 524703)
[PASS] test_revert_invalidOutcomeIndex_aliased() (gas: 33358)
[PASS] test_revert_invalidOutcomeIndex_max(uint256) (runs: 256, μ: 30344, ~: 30336)
[PASS] test_revert_invalidOutcomeIndex_two() (gas: 26918)
Suite result: ok. 4 passed; 0 failed; 0 skipped; finished in 389.98ms (388.10ms CPU time)

Ran 254 test suites in 406.30ms (2.39s CPU time): 932 tests passed, 0 failed, 0 skipped (932 total tests)
```

Code Coverage

The test suite's coverage of the in scope contracts can be obtained by running the following command:

```
forge coverage --no-match-coverage "test|EOA|mocks|legacy|dev|collateral|Quorum|Lz|Ccip|adapters"
```

File	% Lines	% Statements	% Branches	% Funcs
src/abstract/ERC1155TokenReceiver.sol	100.00% (7/7)	100.00% (6/6)	100.00% (0/0)	100.00% (3/3)
src/auth/InitializableRoles.sol	100.00% (18/18)	100.00% (9/9)	100.00% (0/0)	100.00% (9/9)
src/auth/Roles.sol	100.00% (32/32)	100.00% (16/16)	100.00% (0/0)	100.00% (16/16)
src/bridge/abstract/BridgeBase.sol	100.00% (123/123)	100.00% (137/137)	92.50% (37/40)	100.00% (22/22)
src/exchange/Exchange.sol	99.02% (507/512)	96.10% (591/615)	81.90% (86/105)	97.01% (65/67)
src/libraries/BridgePayloads.sol	100.00% (6/6)	100.00% (6/6)	100.00% (0/0)	100.00% (3/3)
src/libraries/Ids.sol	100.00% (43/43)	100.00% (49/49)	100.00% (2/2)	100.00% (19/19)
src/modules/BinaryModule.sol	100.00% (19/19)	100.00% (24/24)	100.00% (2/2)	100.00% (6/6)
src/modules/CombinatorialModule.sol	98.78% (406/411)	99.15% (464/468)	88.70% (102/115)	97.78% (44/45)
src/modules/NegRiskModule.sol	100.00% (98/98)	100.00% (121/121)	92.00% (23/25)	100.00% (16/16)
src/modules/abstract/BaseModule.sol	100.00% (47/47)	100.00% (47/47)	81.82% (9/11)	100.00% (10/10)

File	% Lines	% Statements	% Branches	% Funcs
src/modules/abstract/OracleModule.sol	92.31% (24/26)	93.75% (15/16)	100.00% (6/6)	90.00% (9/10)
src/modules/migration/BaseMigrationMixin.sol	100.00% (80/80)	100.00% (98/98)	96.15% (25/26)	100.00% (10/10)
src/modules/migration/BinaryMigrationMixin.sol	96.67% (29/30)	92.00% (23/25)	85.71% (6/7)	100.00% (11/11)
src/modules/migration/NegRiskMigrationMixin.sol	96.00% (48/50)	94.64% (53/56)	87.50% (14/16)	91.67% (11/12)
src/oracle/OracleAggregator.sol	99.39% (163/164)	98.95% (189/191)	82.61% (57/69)	100.00% (19/19)
src/oracle/abstract/OracleModuleBase.sol	100.00% (11/11)	100.00% (6/6)	100.00% (4/4)	100.00% (5/5)
src/oracle/libraries/OptimisticOraclePayoutLib.sol	100.00% (8/8)	100.00% (11/11)	100.00% (6/6)	100.00% (1/1)
src/oracle/mixins/Auth.sol	100.00% (12/12)	100.00% (6/6)	100.00% (0/0)	100.00% (6/6)
src/oracle/mixins/Pausable.sol	100.00% (8/8)	100.00% (5/5)	100.00% (2/2)	100.00% (3/3)
src/oracle/modules/OptimisticOracleModule.sol	100.00% (81/81)	98.92% (92/93)	92.31% (12/13)	100.00% (14/14)
src/oracle/modules/reporters/ChainlinkReporterModule.sol	100.00% (64/64)	100.00% (67/67)	100.00% (14/14)	100.00% (14/14)
src/positionManager/PositionManager.sol	94.29% (165/175)	94.67% (160/169)	85.29% (29/34)	100.00% (17/17)
src/routers/BridgeRouter.sol	100.00% (18/18)	100.00% (11/11)	100.00% (0/0)	100.00% (7/7)
src/routers/CtfRouter.sol	100.00% (35/35)	100.00% (42/42)	100.00% (3/3)	100.00% (6/6)
src/routers/Router.sol	100.00% (57/57)	100.00% (66/66)	100.00% (2/2)	100.00% (9/9)
src/utils/AutoRedeemer.sol	100.00% (98/98)	98.32% (117/119)	100.00% (18/18)	100.00% (6/6)
Total	98.84% (2207/2233)	98.06% (2431/2479)	88.27% (459/520)	98.63% (361/366)

Changelog

- 2026-05-06 - Initial report
- 2026-05-18 - Final report
- 2026-05-26 - Updated final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 1300 audits and secured over $500 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Solana, Canton, Polymarket, PancakeSwap, Virtuals Protocol, Wayfinder, Lido, TrustWallet, Alchemy, World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Solana, Polygon, TON, Cardano, Binance Smart Chain, Avalanche, Arbitrum, Flow
- DeFi: Lido, Ethena, Compound, Curve, Venus, PancakeSwap, Polymarket
- Infra/Staking: TrustWallet, Alchemy, Liquid Collective, Kiln, Galxe, API3, ssv.network, Luganodes
- Immersive: Virtuals Protocol, Wayfinder, OpenSea, Square Enix, Parallel, Camp, Decentraland
- Institutional: Visa, Circle, Revolut, Anthropic, OpenAI, Canton, Republic, Arkham, Sequoia

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.